A Modern Multi-Core Processor

Instructor: Jeongseob Ahn (안정섭)

E-mail: jsahn@csl.korea.ac.kr
Hompage: https://jeongseob.github.io

Slide source: CS149 Stanford University by Prof. Kayvon Fatahalian

What is a program?

- From a processor's perspective
  - A program is just a list of processor instructions

```
int main(int argc, char** argv) {
    int x = 1;

    for (int i=0; i<10; i++) {
        x = x + x;
    }

    printf("%d\n", x);

    return 0;
}
```

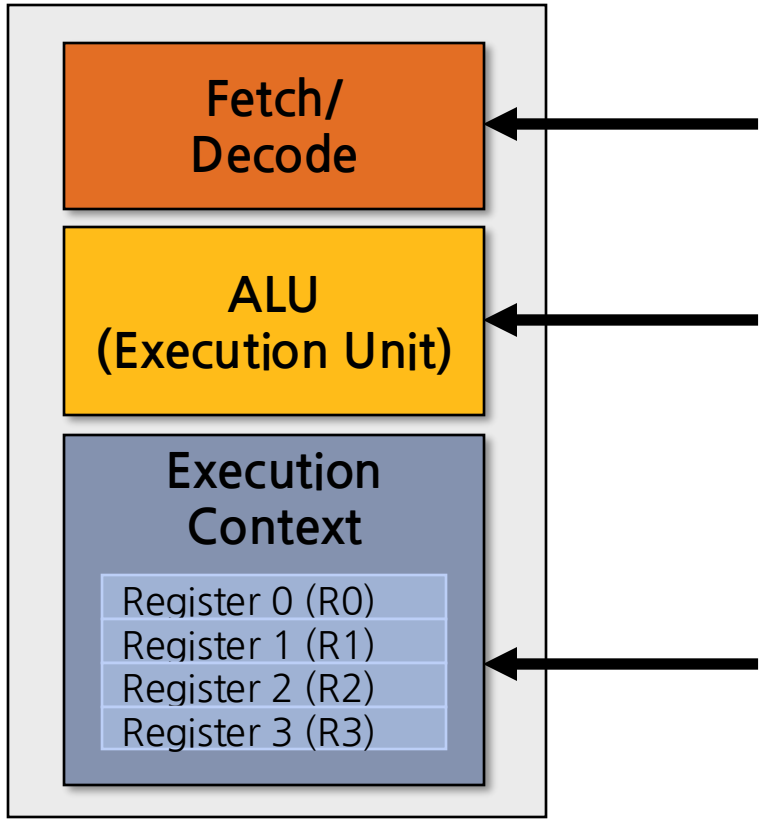
Compile code

```
_main:
100000f10:    pushq %rbp
100000f11:    movq %rsp, %rbp
100000f14:    subq $32, %rsp
100000f18:    movl $0, -4(%rbp)
100000f1f:    movl %edi, -8(%rbp)
100000f22:    movq %rsi, -16(%rbp)
100000f26:    movl $1, -20(%rbp)
100000f2d:    movl $0, -24(%rbp)
100000f34:    cmpl $10, -24(%rbp)
100000f38:    jge 23 <_main+0x45>
100000f3e:    movl -20(%rbp), %eax
100000f41:    addl -20(%rbp), %eax
100000f44:    movl %eax, -20(%rbp)
100000f47:    movl -24(%rbp), %eax
100000f4a:    addl $1, %eax
100000f4d:    movl %eax, -24(%rbp)
100000f50:    jmp -33 <_main+0x24>
100000f55:    leaq 58(%rip), %rdi
100000f5c:    movl -20(%rbp), %esi
100000f5f:    movb $0, %al
100000f61:    callq 14
100000f66:    xorl %esi, %esi
100000f68:    movl %eax, -28(%rbp)
100000f6b:    movl %esi, %eax
100000f6d:    addq $32, %rsp
100000f71:    popq %rbp
100000f72:    rets
```

What does a processor do?

A processor execute instructions

A Very Simple Processor



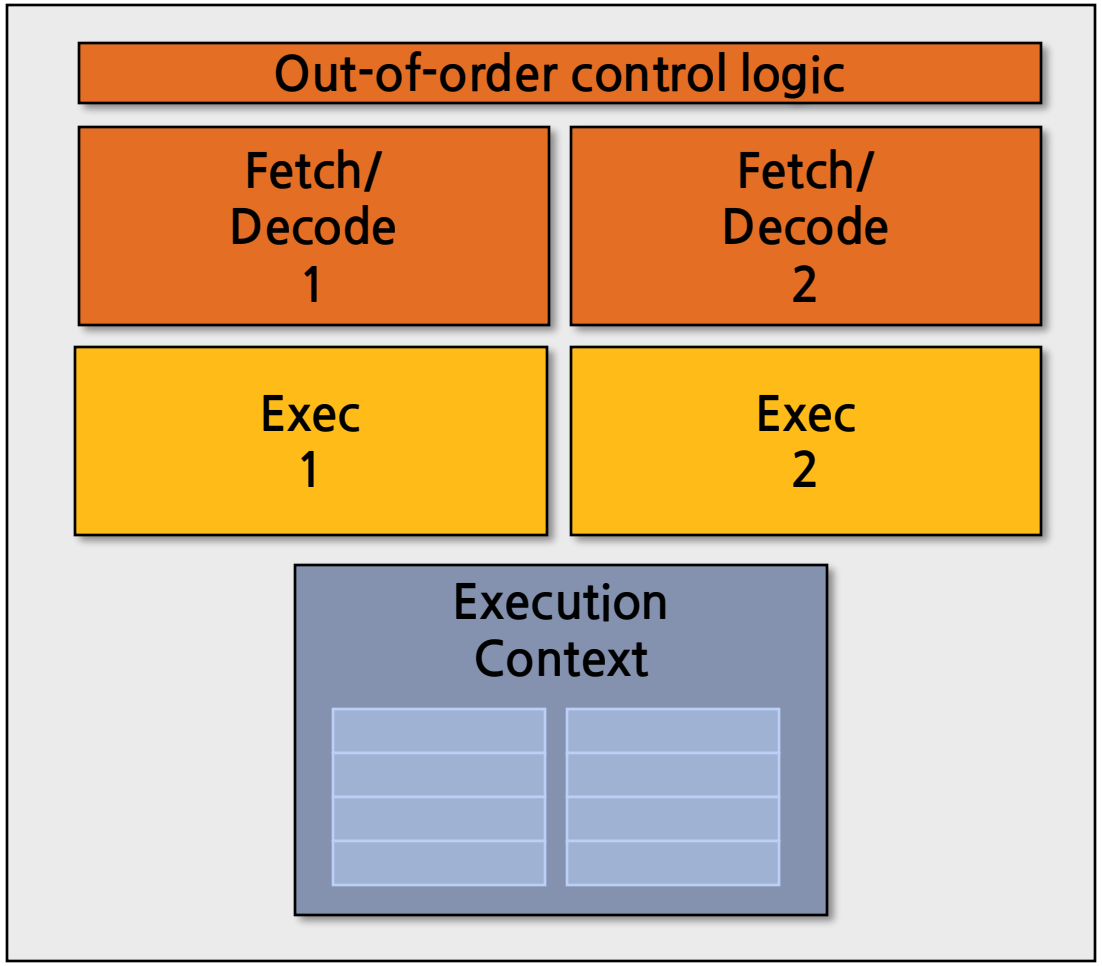
Determine what instructions to run next

Execution unit: performs the operation described by an instruction, which may modify values in the processor's registers or the computer's memory

Registers: maintain program state: store value of variables used as inputs and outputs to operations

Superscalar processor

This processor can decode and execute up to two instructions per clock



Superscalar execution: processor automatically finds independent instructions in an instruction sequence and can execute them in parallel on multiple execution units.

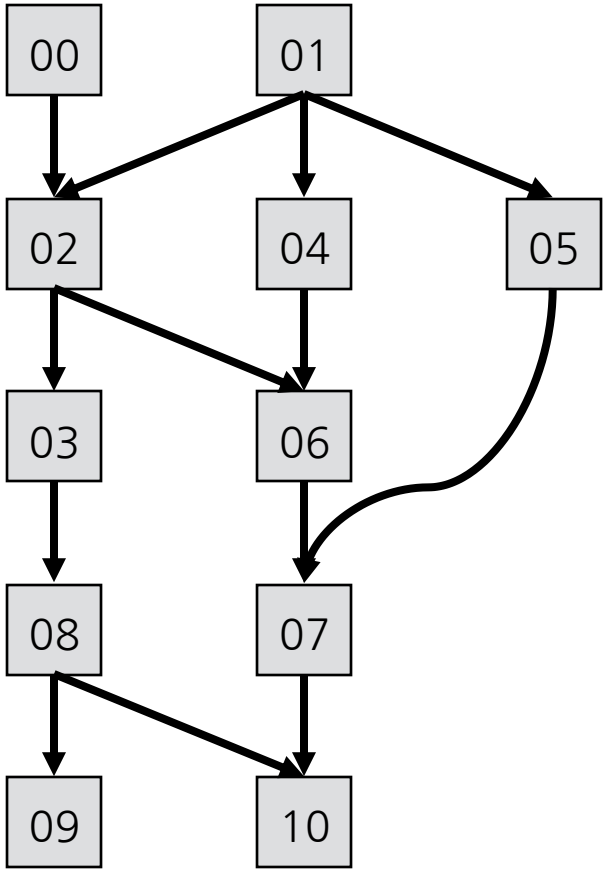
A more complex example

Program (sequence of instructions)

PC	Instruction	
00	a = 2	
01	b = 4	
02	tmp2 = a + b	// 6
03	tmp3 = tmp2 + a	// 8
04	tmp4 = b + b	// 8
05	tmp5 = b * b	// 16
06	tmp6 = tmp2 + tmp4	// 14
07	tmp7 = tmp5 + tmp6	// 30
08	if (tmp3 > 7)	
09	print tmp3	
	else	
10	print tmp7	

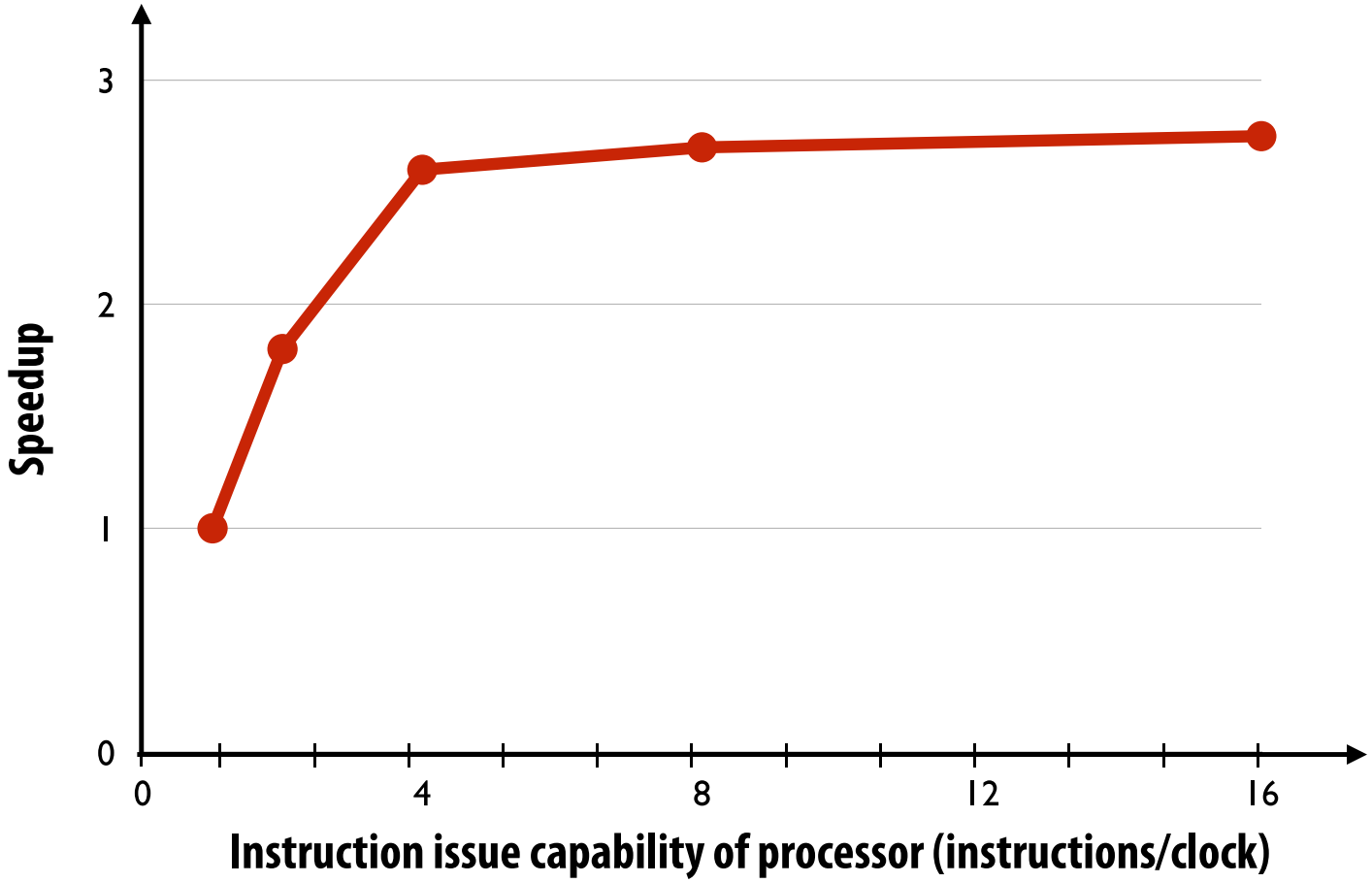
value during execution

Instruction dependency graph



Diminishing returns of superscalar execution

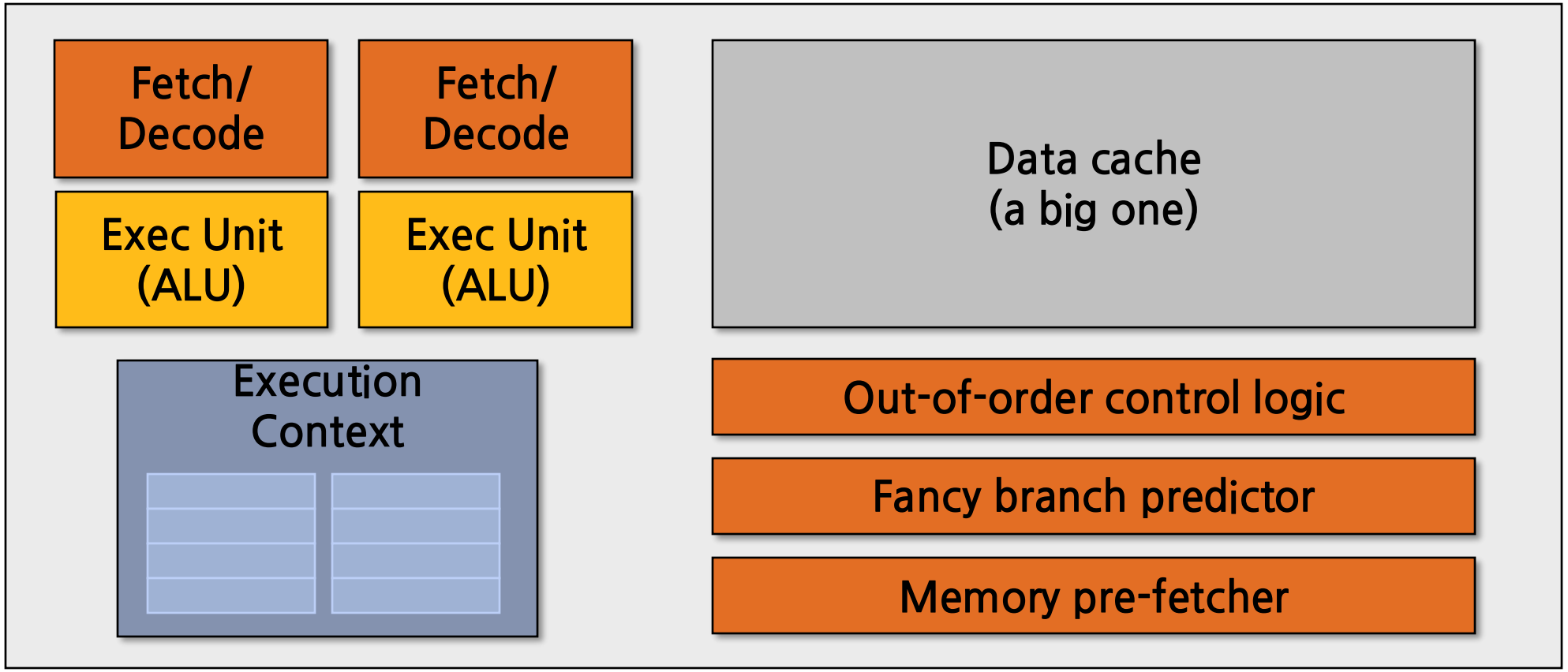
Most available ILP is exploited by a processor capable of issuing four instructions per clock
(Little performance benefit from building a processor that can issue more)



Source: Culler & Singh (data from Johnson 1991)

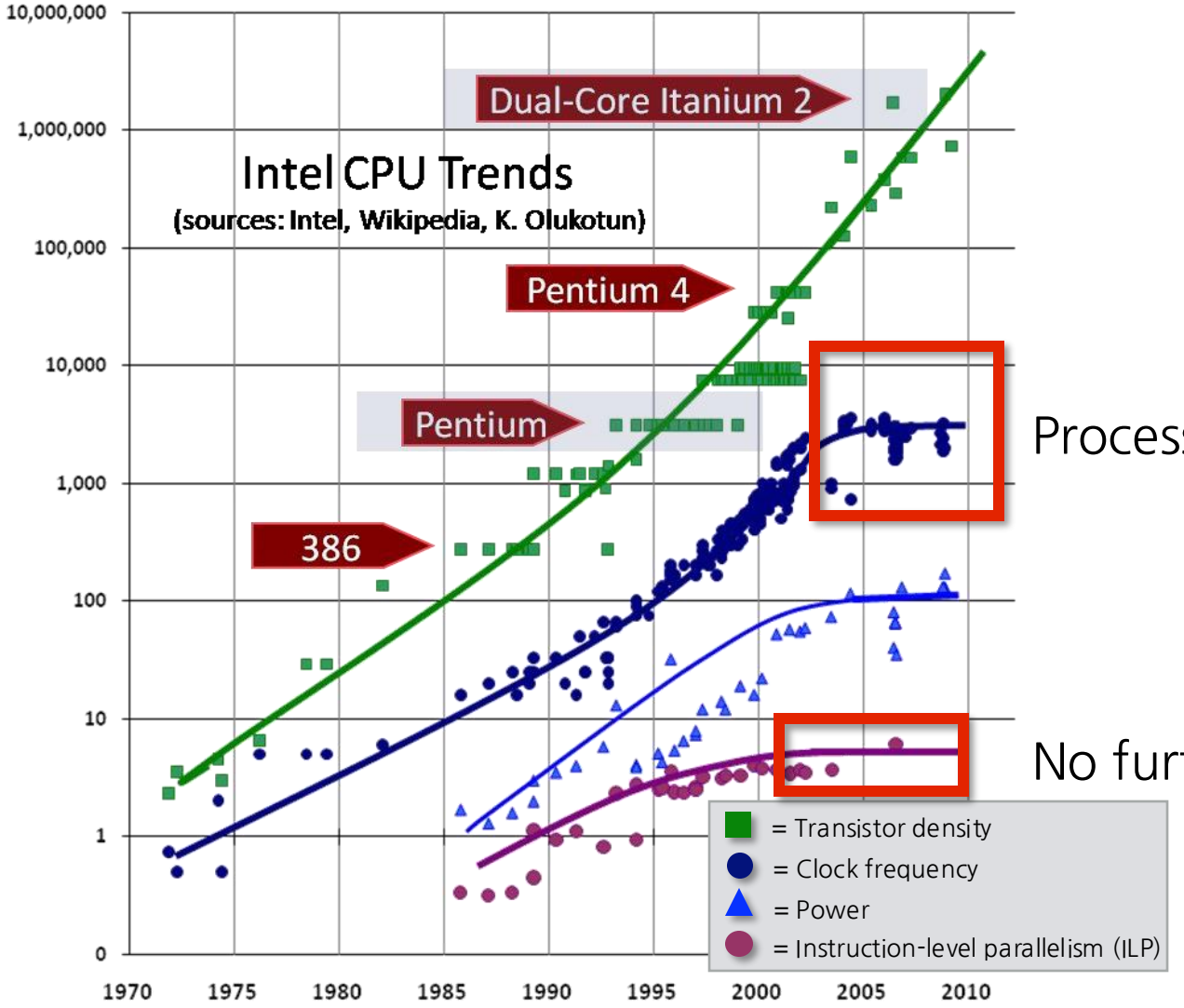
Pre multi-core era processor

Majority of chip transistors used to perform operations that help make a single instruction stream run fast



More transistors = larger cache, smarter out-of-order logic, smarter branch predictor, etc.

ILP tapped out + end of frequency scaling



Processor clock rate stops increasing

No further benefit from ILP

Image credit: "The free Lunch is Over" by Herb Sutter, Dr. Dobbs 2005

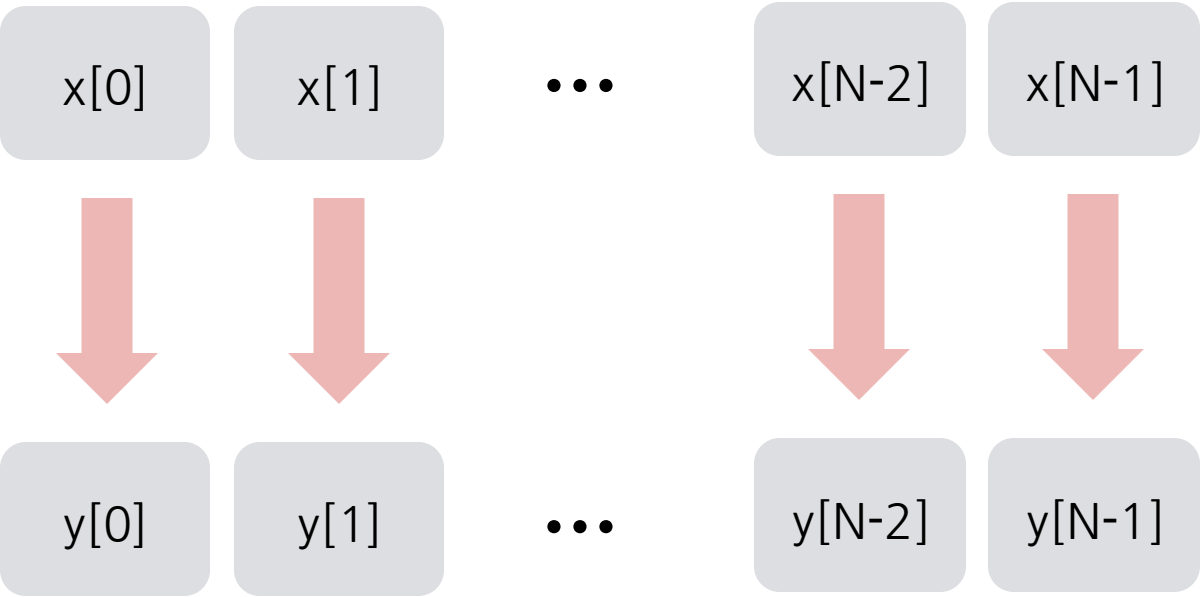
Today's example program

```
void sinx(int N, int terms, float* x, float* y)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6; //3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

Compute $\sin(x)$ using Taylor expansion:

$\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots$

for each element of an array of N floating-point numbers



Compile program

```
void sinx(int N, int terms, float* x, float* y)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6; //3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

compiler

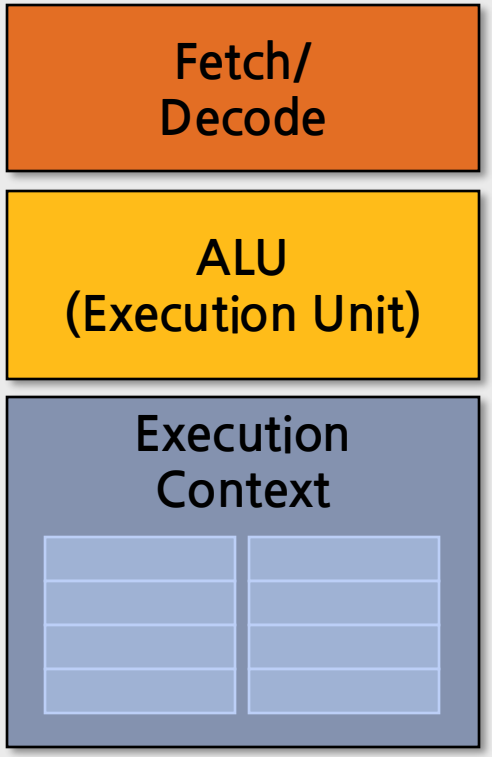
Compiled instruction stream
(scalar instructions)

x[i]

```
ld r0, addr[r1]
mul r1, r0, r0
mul r1, r1, r0
...
...
...
...
...
...
st addr[r2], r0
```

y[i]

Multi-core era processor

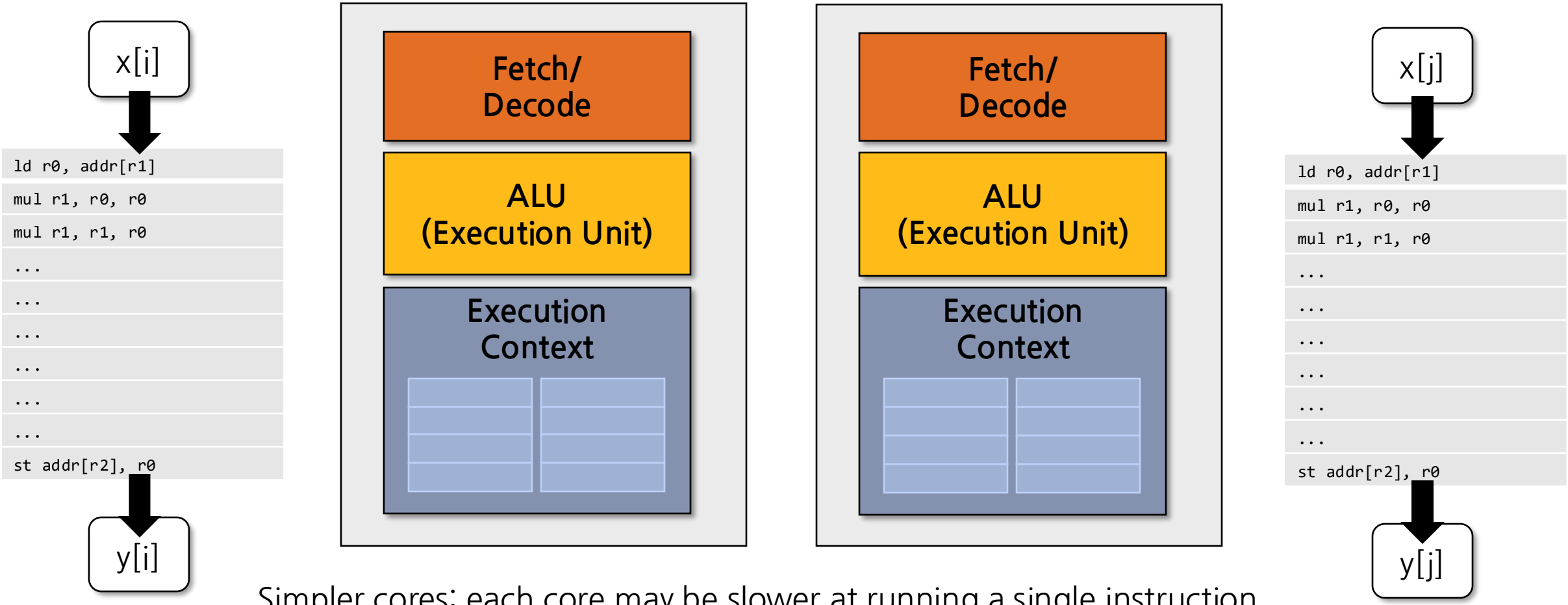


Idea #1:

Rather than use transistors to increase sophistication of processor logic that accelerates a single instruction stream (e.g., out-of-order and speculative operations)

Use increasing transistor count to add more cores to the processor

Two cores: compute two elements in parallel



Simpler cores: each core may be slower at running a single instruction stream than our original "fancy" core (e.g., 25% slower)

But there are now two cores: 2 x 0.75 = 1.5 (potential for speedup!)

But our program expresses no parallelism

```
void sinx(int N, int terms, float* x, float* y)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6; //3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

This C program will compile to an instruction stream that runs as one thread on one processor core.

If each of the simpler processor cores was 25% slower than the original single complicated one, our program now runs 25% slower than before.

Example: expressing parallelism using pthreads

```
typedf struct {
    int N;
    int terms;
    float* x;
    float* result;
} my_args;

void parallel_sinx(int N, int terms, float* x, float* result)
{
    pthread_t thread_id;
    my_args args;

    args.N = N/2;
    args.terms = terms;
    args.x = x;
    args.result = result;

    pthread_create(&thread_id, NULL, my_thread_start, &args); // launch thread
    sinx(N - args.N, terms, x + args.N, result + args.N); // do work
    pthread_join(thread_id, NULL);
}

void my_thread_start(void* thread_arg)
{
    my_args* thread_args = (my_args*)thread_arg;
    sinx(args->N, args->terms, args->x, args->result); // do work
}
```

```
void sinx(int N, int terms, float* x, float* y)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6; //3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

Data-parallel expression

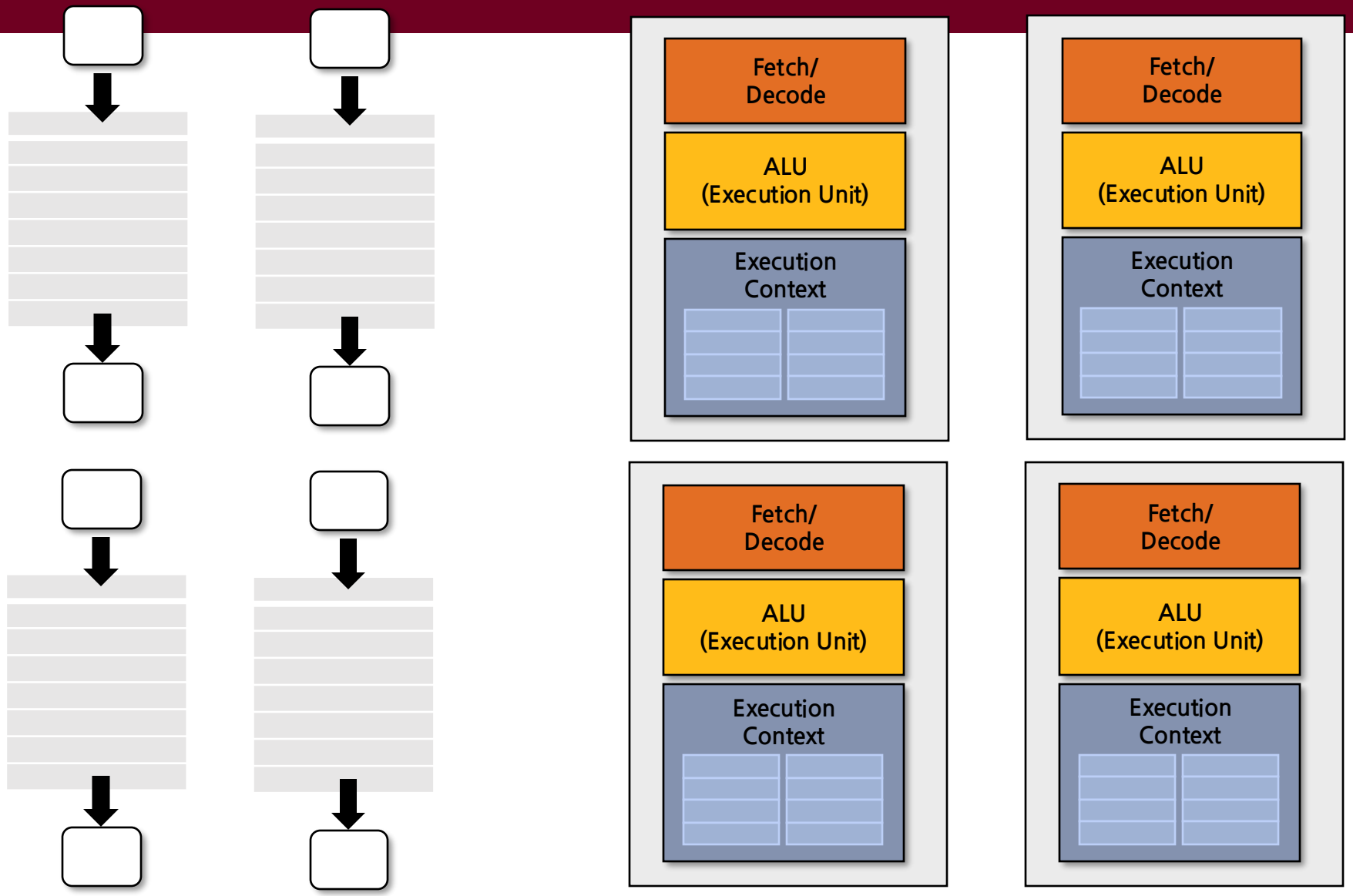
```
void sinx(int N, int terms, float* x, float* y)
{
    // declares that loop iterations are independent
    forall (int i from 0 to N)
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6; //3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

(in a fictitious data-parallel language)

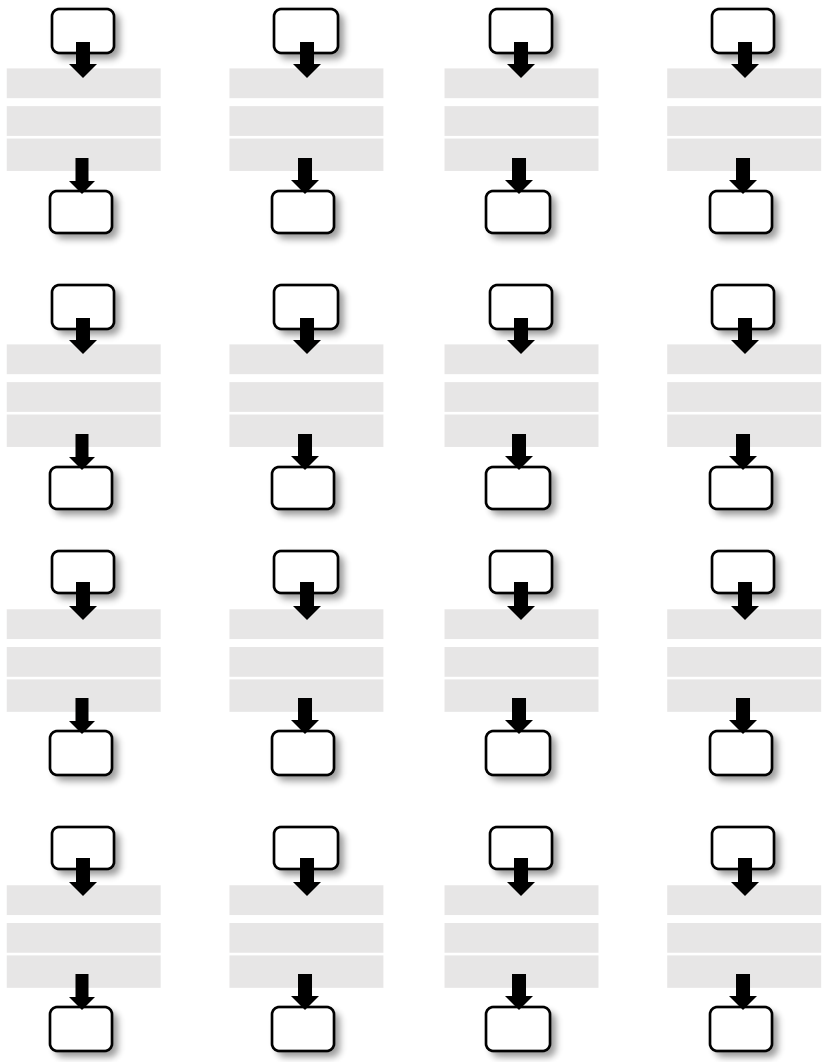
In this code, loop iterations are declared by the programmer to be independent (see the 'forall')

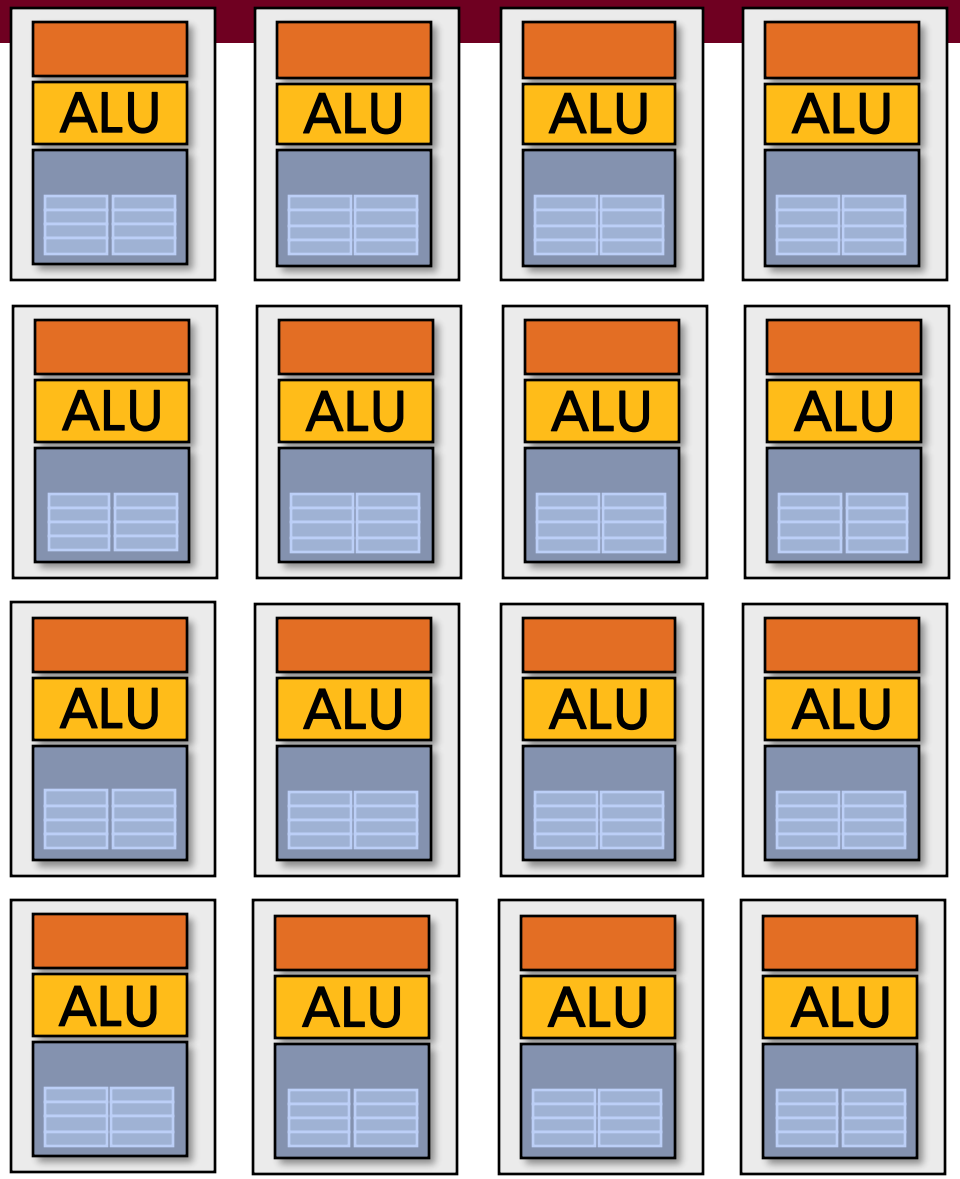
With this information, you could imagine how a compiler might automatically generate parallel threaded code for you.

Four cores: compute four elements in parallel



Sixteen cores: computes sixteen elements in parallel





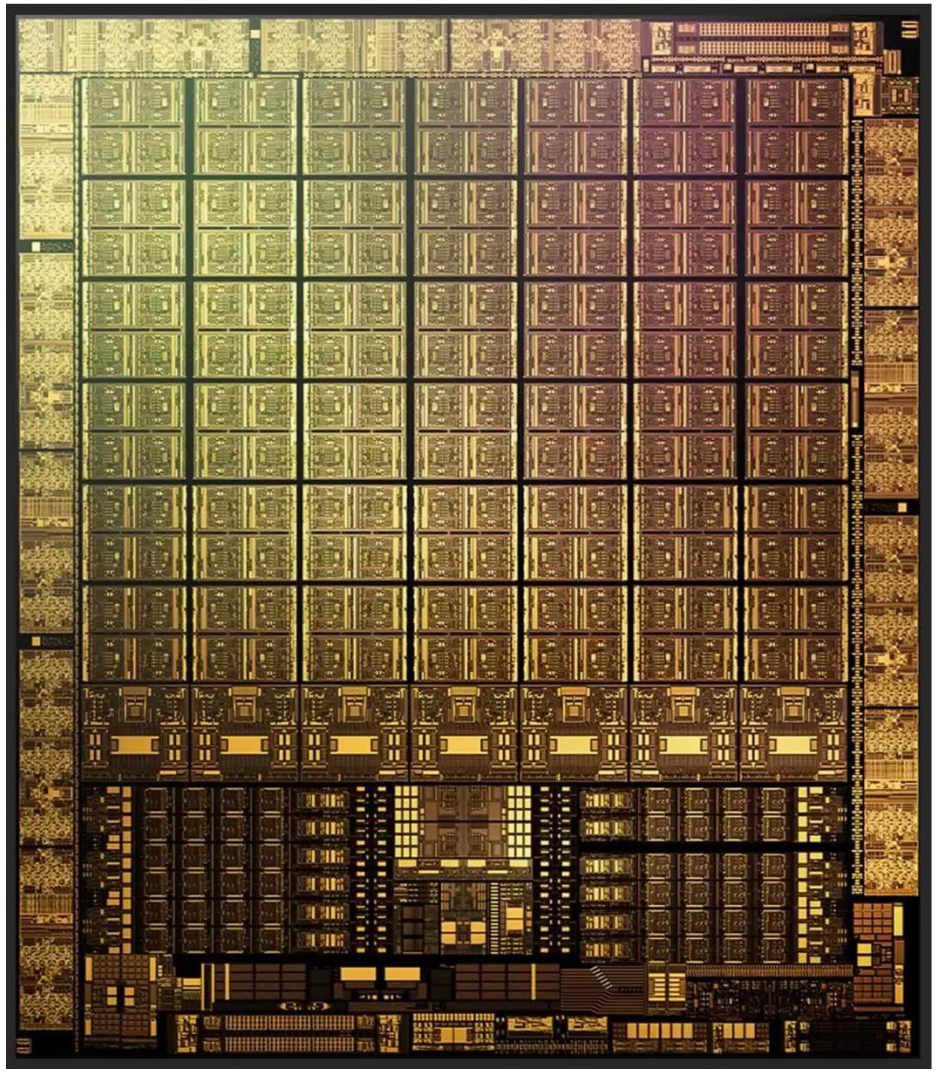
Sixteen cores, sixteen simultaneous instruction streams

Example: multi-core CPU

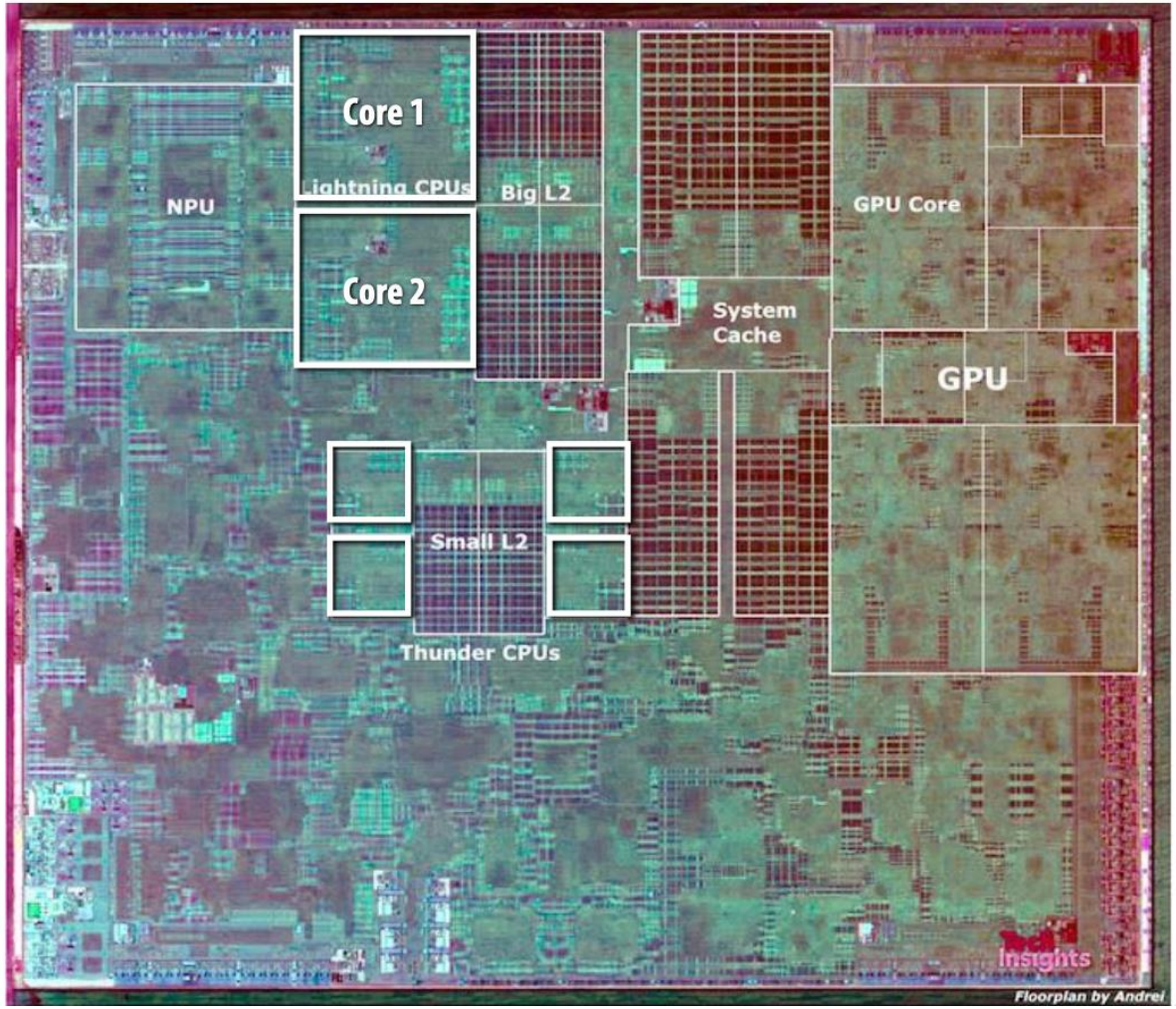
- Intel “Comet Lake” 10th Generation Core i9 10-core CPU (2020)

Multi-core GPU

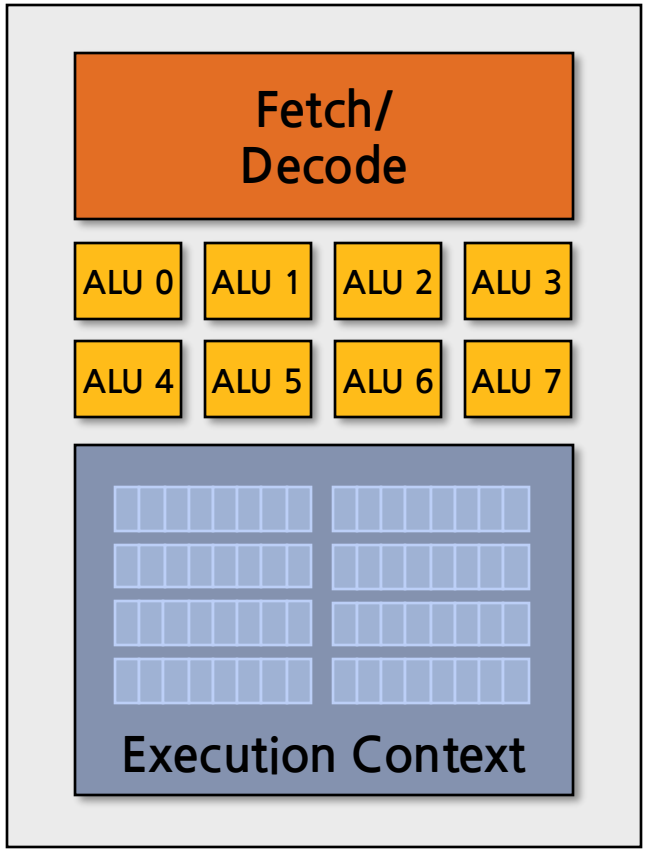
- NVIDIA Amphere GPU
- 84 “SM” cores



Apple A13: Two “big” cores + four “small” cores



Add ALUs to increase compute capability



Idea #2:
Amortize cost/complexity of managing an instruction stream across many ALUs

SIMD processing

Single instruction, multiple data

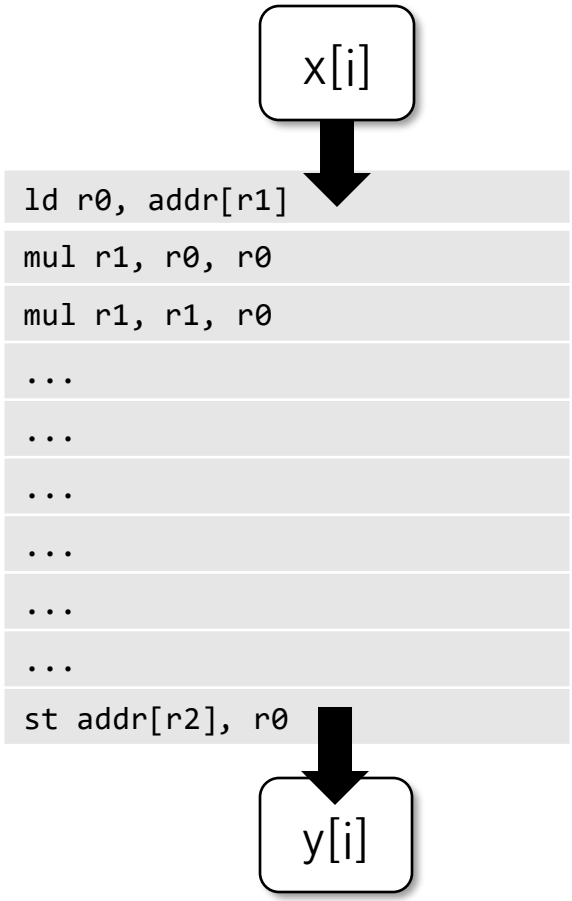
Same instruction broadcast to all ALUs
This operation is executed in parallel on all ALUs

Recall our original scalar program

```
void sinx(int N, int terms, float* x, float* y)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6; //3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

Original compiled program:

Processes one array element using scalar instructions on scalar register (e.g., 32-bit floats)



Vector program (using AVX intrinsics)

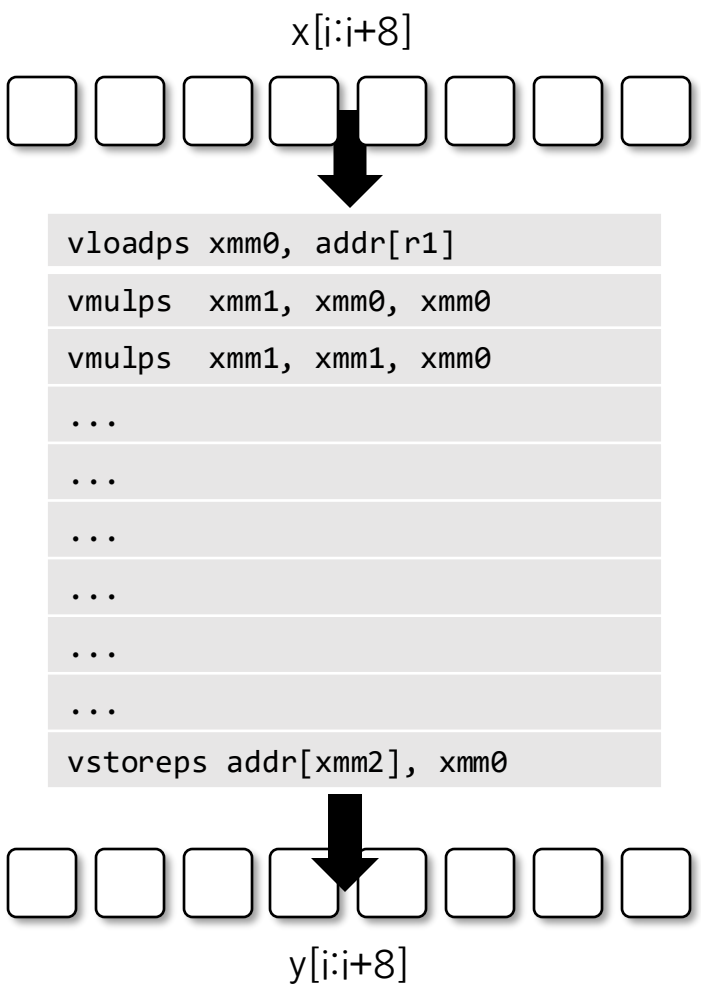
```
#include <immintrin.h>

void sinx(int N, int terms, float* x, float* y)
{
    float three_fact = 6; // 3!
    float denom_base;

    for (int i=0; i<N; i+=8)
    {
        __m256 origx = _mm256_load_ps(&x[i]);
        __m256 value = origx;
        __m256 numer = _mm256_mul_ps(origx, _mm256_mul_ps(origx, origx));
        __m256 denom = _mm256_broadcast_ss(&three_fact);
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            // value += sign * numer / denom
            __m256 tmp = _mm256_div_ps(_mm256_mul_ps(_mm256_set1_ps(sign), numer), denom);
            value = _mm256_add_ps(value, tmp);
            numer = _mm256_mul_ps(numer, _mm256_mul_ps(origx, origx));
            denom_base = (2*j+2) * (2*j+3);
            denom = _mm256_mul_ps(denom, _mm256_broadcast_ss(&denom_base));
            sign *= -1;
        }
        _mm256_store_ps(&y[i], value);
    }
}
```

Intrinsic datatypes and functions available to C programmers

Intrinsic functions operate on vectors of eight 32-bit values (e.g., vector of 8 floats)

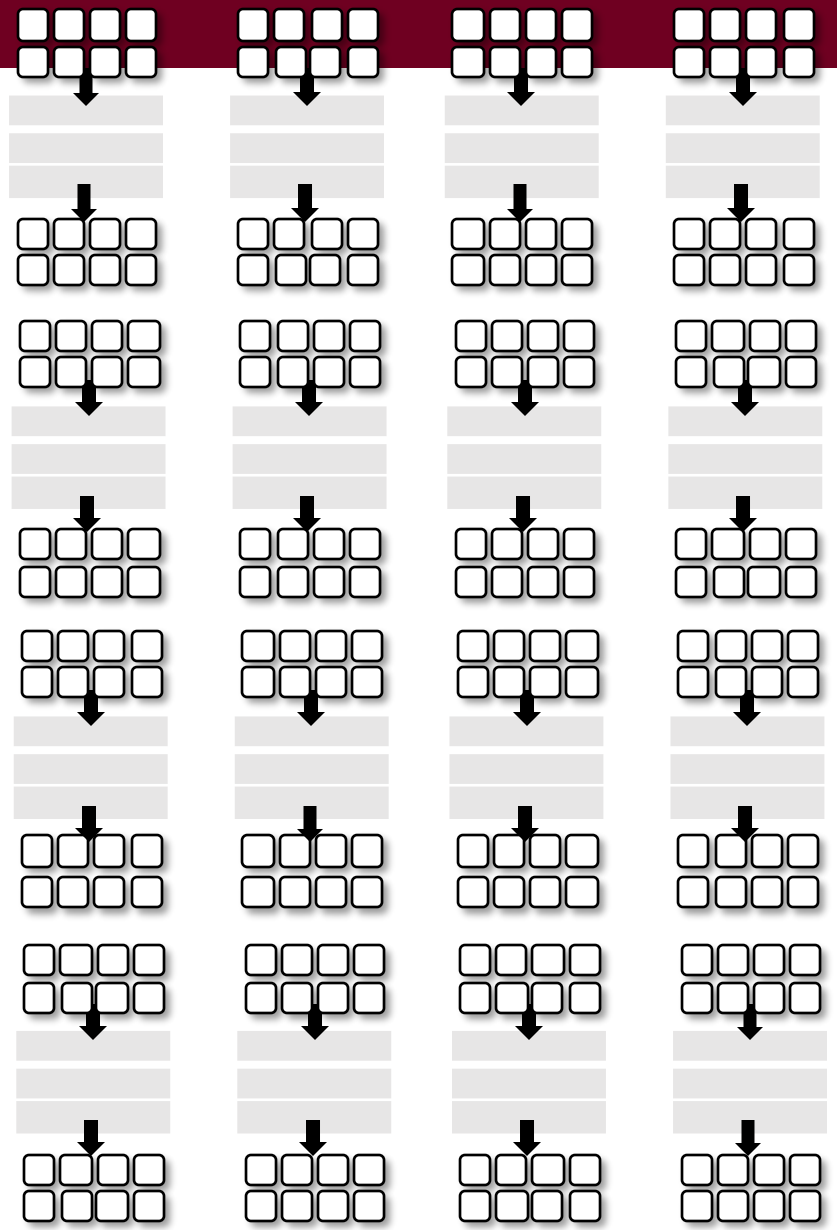
Vector program (using AVX intrinsics)

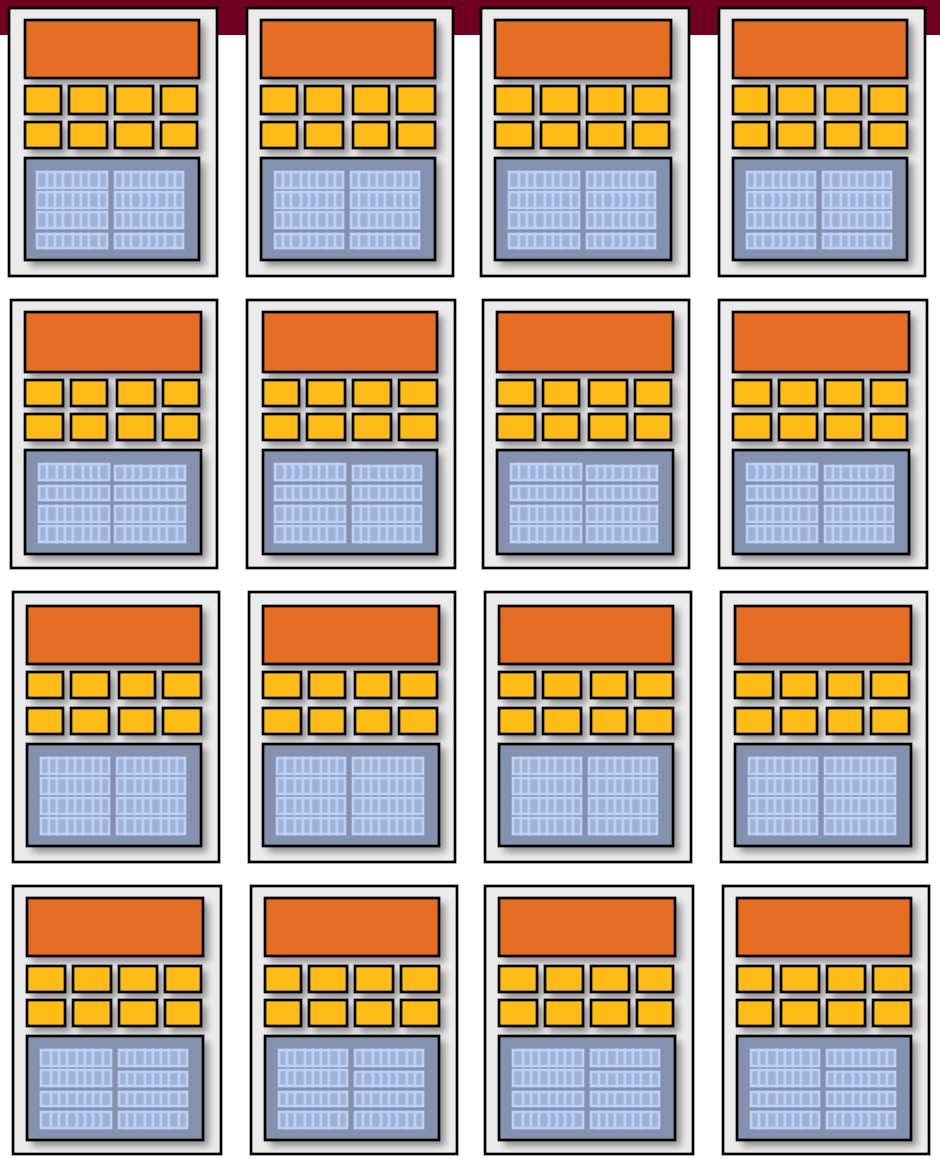
```c
#include <immintrin.h>

void sinx(int N, int terms, float* x, float* y)
{
    float three_fact = 6; // 3!
    float denom_base;

    for (int i=0; i<N; i+=8)
    {
        __m256 origx = _mm256_load_ps(&x[i]);
        __m256 value = origx;
        __m256 numer = _mm256_mul_ps(origx, _mm256_mul_ps(origx, origx));
        __m256 denom = _mm256_broadcast_ss(&three_fact);
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            // value += sign * numer / denom
            __m256 tmp = _mm256_div_ps(_mm256_mul_ps(_mm256_set1_ps(sign), numer), denom);
            value = _mm256_add_ps(value, tmp);
            numer = _mm256_mul_ps(numer, _mm256_mul_ps(origx, origx));
            denom_base = (2*j+2) * (2*j+3);
            denom = _mm256_mul_ps(denom, _mm256_broadcast_ss(&denom_base));
            sign *= -1;
        }
        _mm256_store_ps(&y[i], value);
    }
}
```



Processes eight array elements simultaneously using vector instructions on 256-bit vector registers

16 SIMD cores: 128 elements in parallel



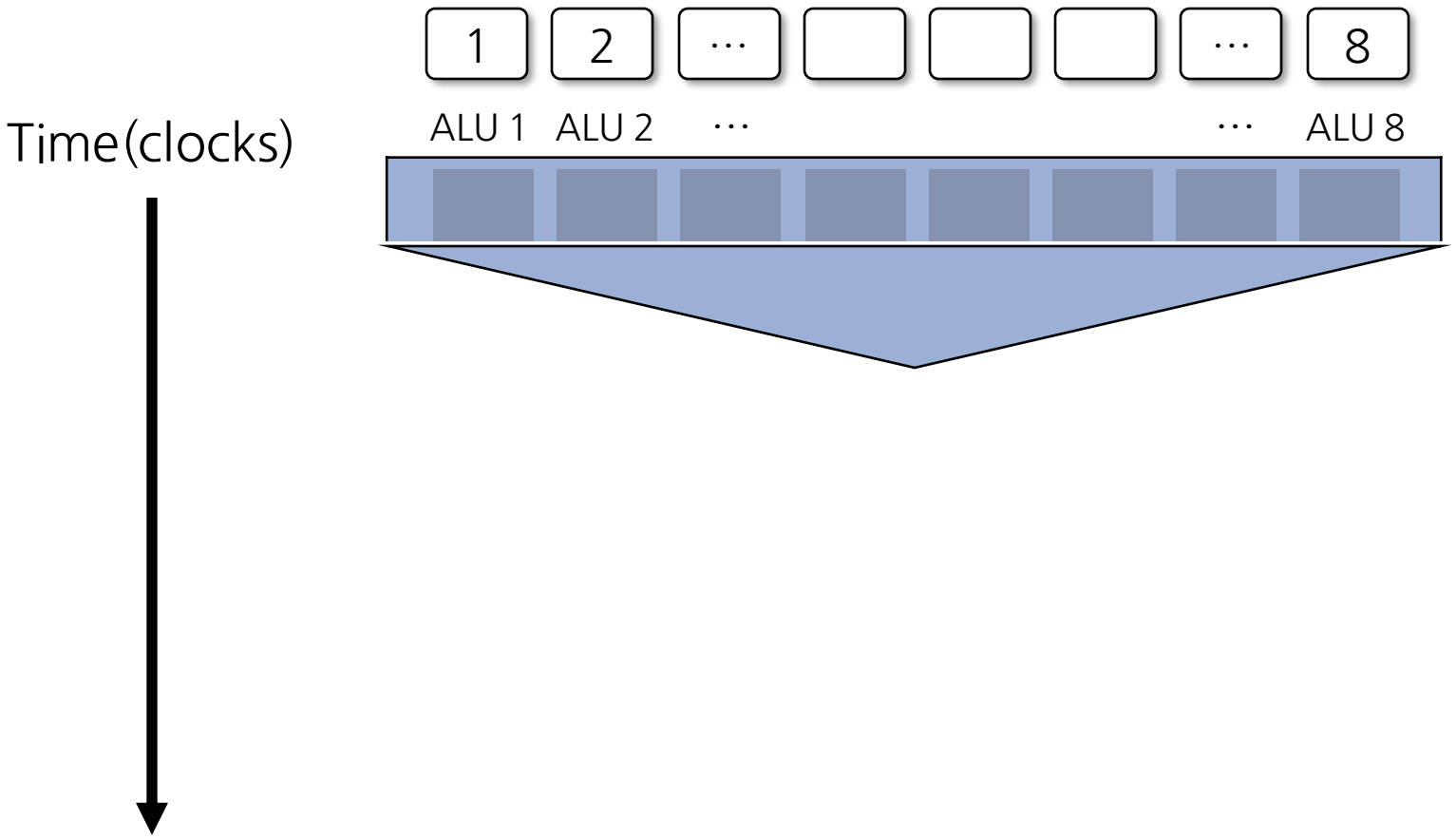


16 cores

128 ALUs

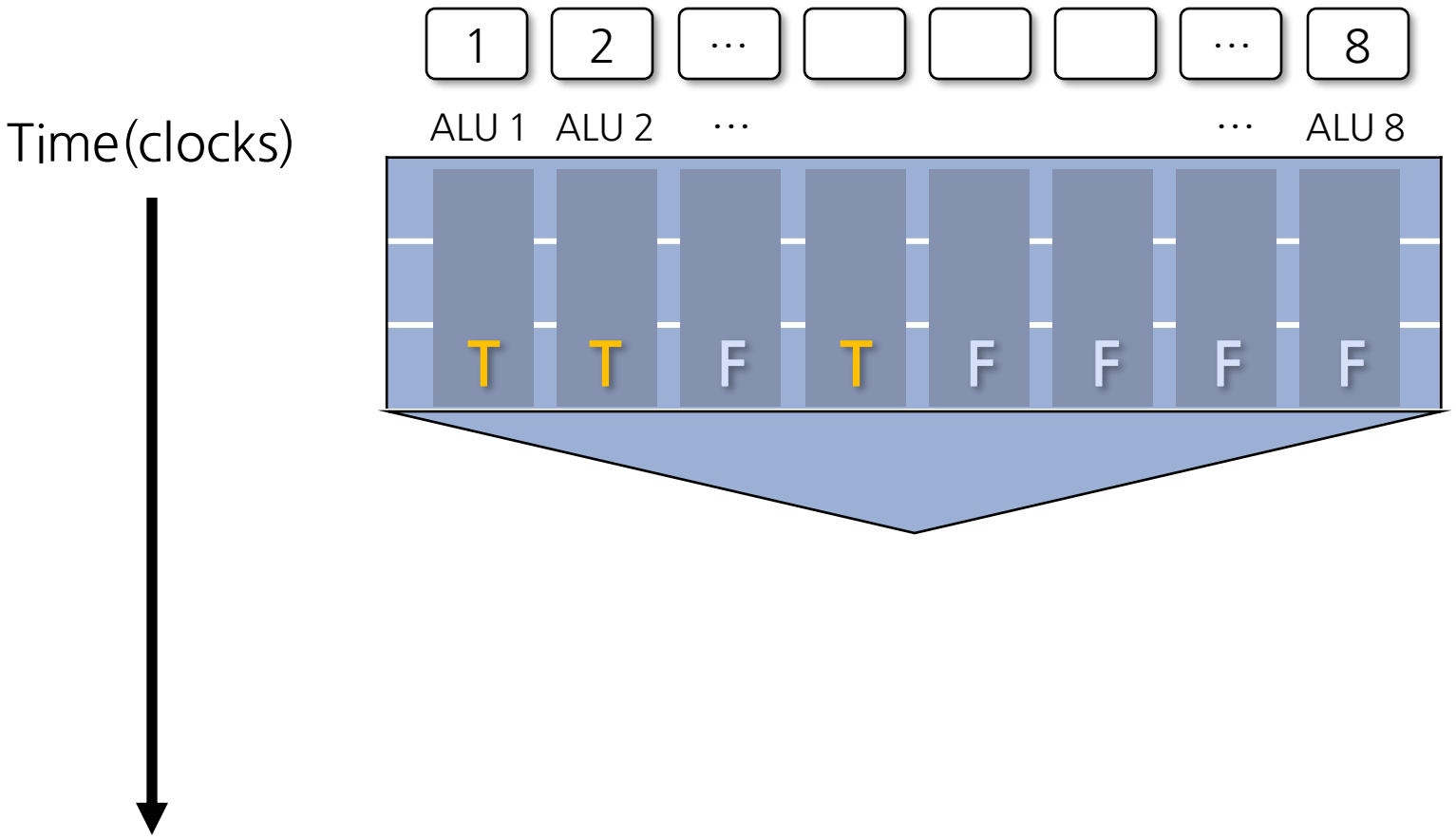
16 simultaneous instruction streams

What about conditional execution?



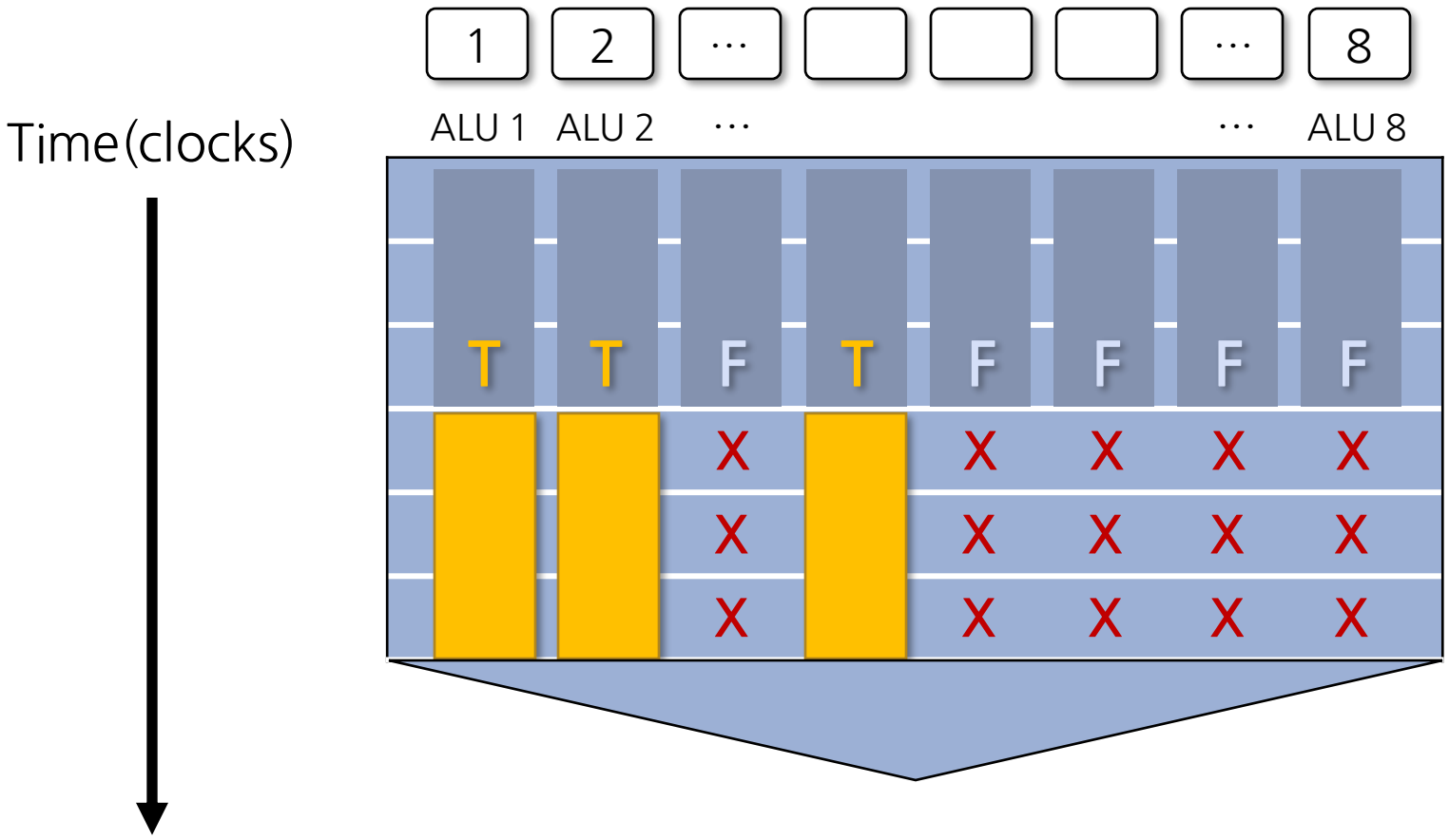

```
forall (int i from 0 to N) {
    float t = x[i];

    <unconditional code>

    if (t > 0.0) {
        t = t * t;
        t = t * 50.0;
        t = t + 100.0;
    } else {
        t = t + 30.0;
        t = t / 10.0;
    }

    <resume unconditional code>
    y[i] = t;
}
```

What about conditional execution?



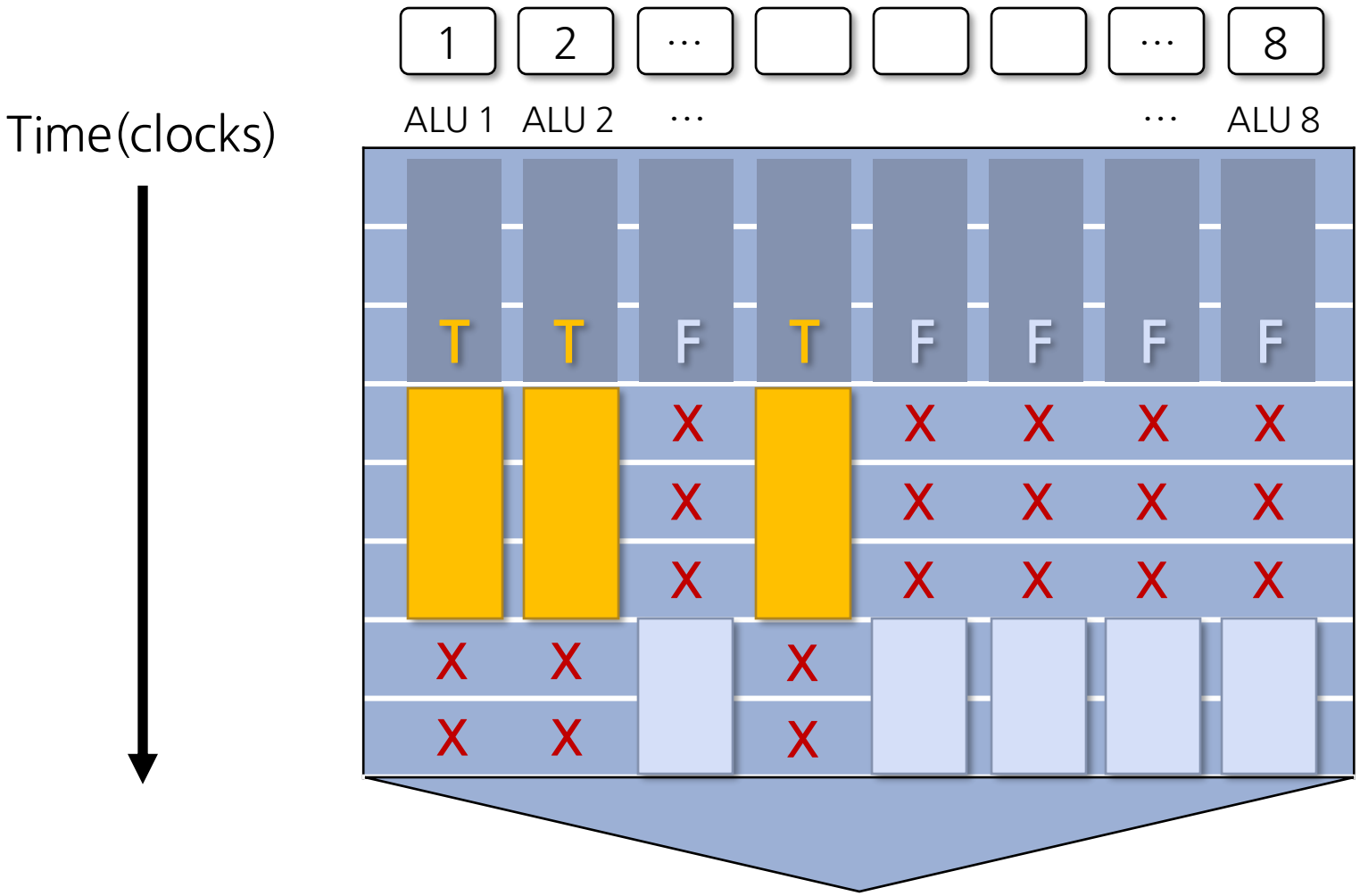

```
forall (int i from 0 to N) {
    float t = x[i];

    <unconditional code>

    if (t > 0.0) {
        t = t * t;
        t = t * 50.0;
        t = t + 100.0;
    } else {
        t = t + 30.0;
        t = t / 10.0;
    }

    <resume unconditional code>
    y[i] = t;
}
```

Mask (discard) output of ALU



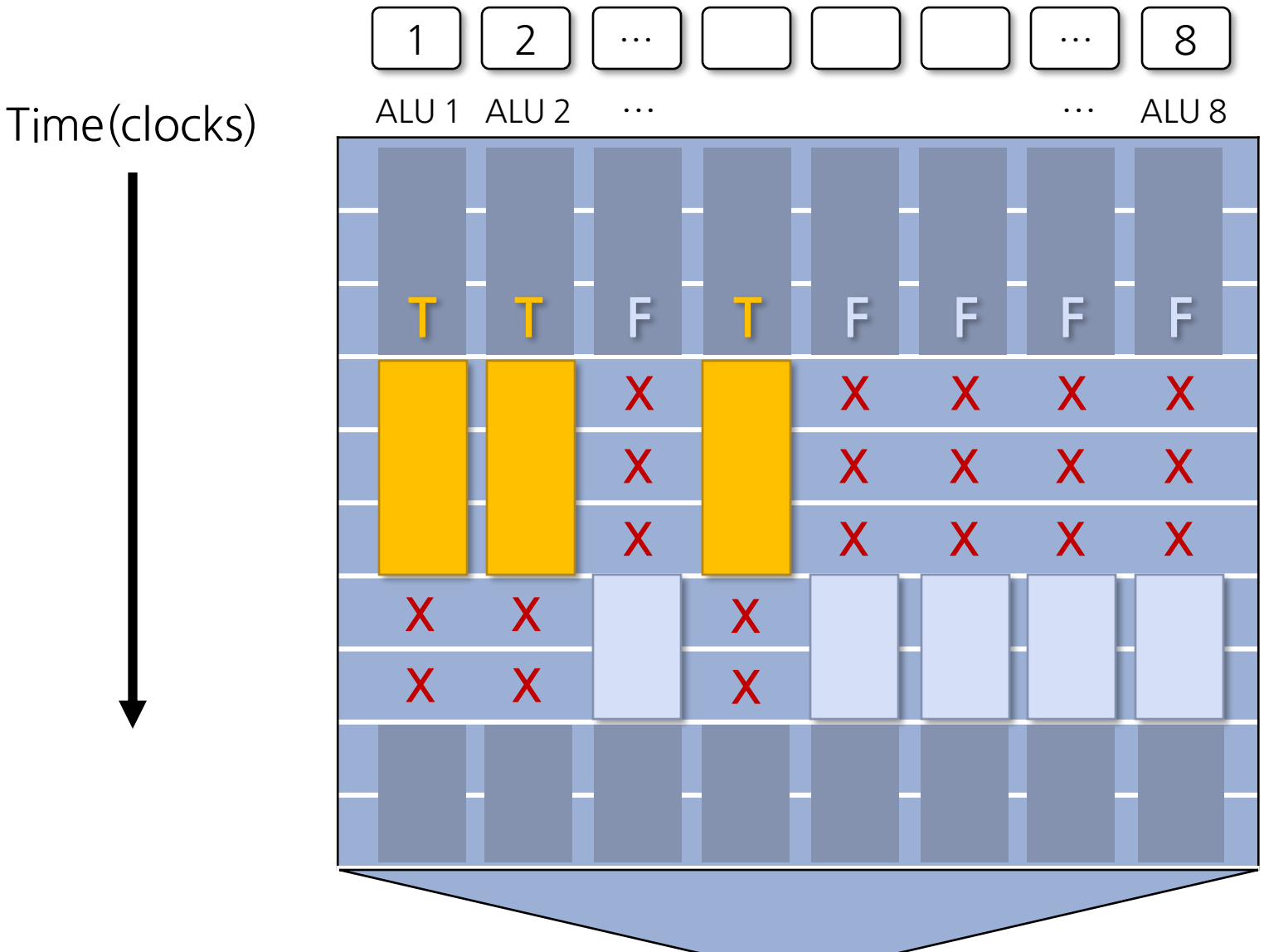

```
forall (int i from 0 to N) {
    float t = x[i];

    <unconditional code>

    if (t > 0.0) {
        t = t * t;
        t = t * 50.0;
        t = t + 100.0;
    } else {
        t = t + 30.0;
        t = t / 10.0;
    }

    <resume unconditional code>
    y[i] = t;
}
```

Mask (discard) output of ALU




```
forall (int i from 0 to N) {
    float t = x[i];

    <unconditional code>

    if (t > 0.0) {
        t = t * t;
        t = t * 50.0;
        t = t + 100.0;
    } else {
        t = t + 30.0;
        t = t / 10.0;
    }

    <resume unconditional code>
    y[i] = t;
}
```

After branch: continue at full performance




```
forall (int i from 0 to N) {
    float t = x[i];

    <unconditional code>

    if (t > 0.0) {
        t = t * t;
        t = t * 50.0;
        t = t + 100.0;
    } else {
        t = t + 30.0;
        t = t / 10.0;
    }

    <resume unconditional code>
    y[i] = t;
}
```

Some common terminology

- Instruction stream coherence (“coherent execution”)
  - Property of a program where the same instruction sequence applies to many data elements
  - Coherent execution IS NECESSARY for SIMD processing resources to be used efficiently
  - Coherent execution IS NOT NECESSARY for efficient parallelization across different cores, since each core has the capability to fetch/decode a different instructions from their thread’s instruction stream
- “Divergent” execution
  - A lack of instruction stream coherence

SIMD execution: modern CPU examples

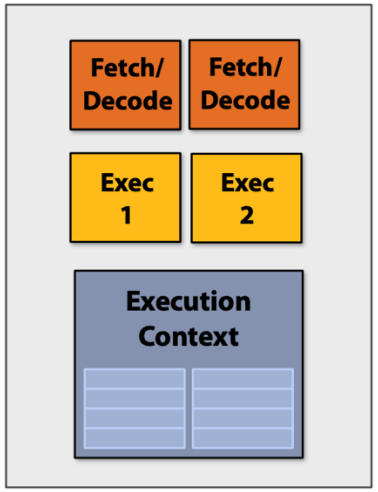
- Intel AVX2 instructions: 256 bit operations: 8x32 bits or 4x64 bits (8-wide float vectors)
- Intel AVX512 instruction: 512 bit operations: 16x32 bits...
- ARM Neon instructions: 128 bit operations: 4x32 bits...
- Instructions are generated by the compiler
  - Parallelism explicitly requested by programmer using intrinsics
  - Parallelism conveyed using parallel language semantics (e.g.,`forall` example)
  - Parallelism inferred by dependency analysis of loops by "auto-vectorizing" compiler
- Terminology: "explicit SIMD": SIMD parallelization is performed at compile time
  - Can inspect program binary and see SIMD instructions (`vstoreps`,`vmulps`, `etc`.)

SIMD execution on many modern GPUs

- “Implicit SIMD”
  - Compiler generates a binary with scalar instructions
  - But N instances of the program are always run together on the processor
  - Hardware (not compiler) is responsible for simultaneously executing the same instruction from multiple program instances on different data on SIMD ALUs
- SIMD width of most modern GPUs ranges from 8 to 32
  - Divergent execution can be a big issue
    (poorly written code might execute at 1/32 the peak capability of the machine!)

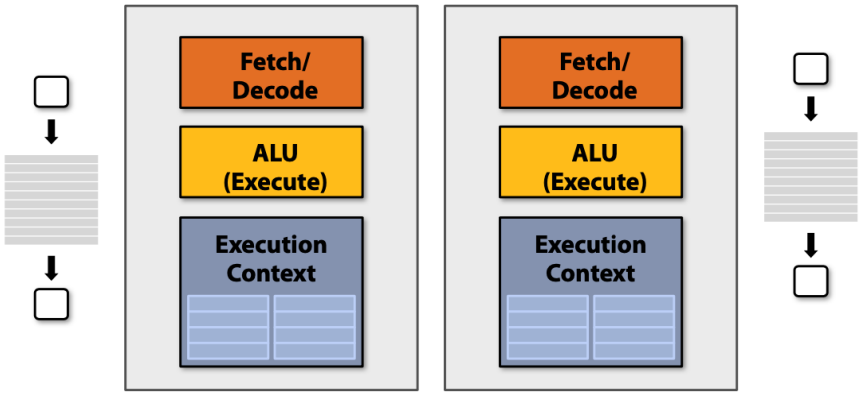
Summary: three different forms of parallel execution

- **Superscalar**: exploit ILP within an instruction stream. Process different instructions from the same instruction stream in parallel (within a core)
  - Parallelism automatically discovered by the hardware during execution
- **SIMD**: multiple ALUs controlled by same instruction (within a core)
  - Efficient for data-parallel workloads: amortize control costs over many ALUs
  - Vectorization done by compiler (explicit SIMD) or at runtime by hardware (implicit SIMD)
- **Multi-core**: use multiple processing cores
  - Provides thread-level parallelism: simultaneously execute a completely different instruction stream on each core
  - Software creates threads to expose parallelism to hardware (e.g., via threading API)

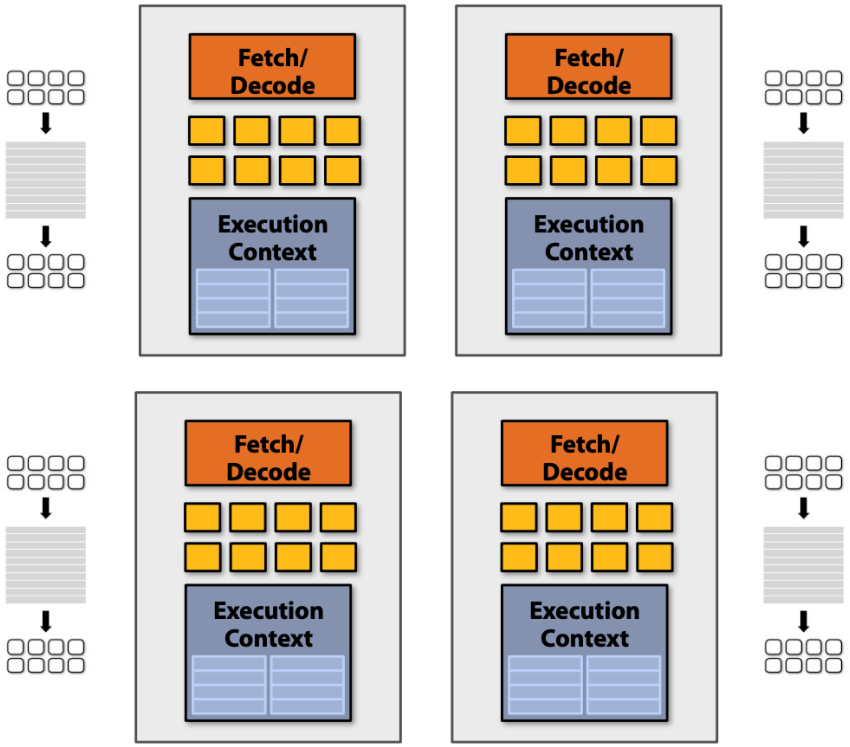


My single core, superscalar processor:
executes up to two instructions per clock from a single instruction stream (if the instructions are independent)

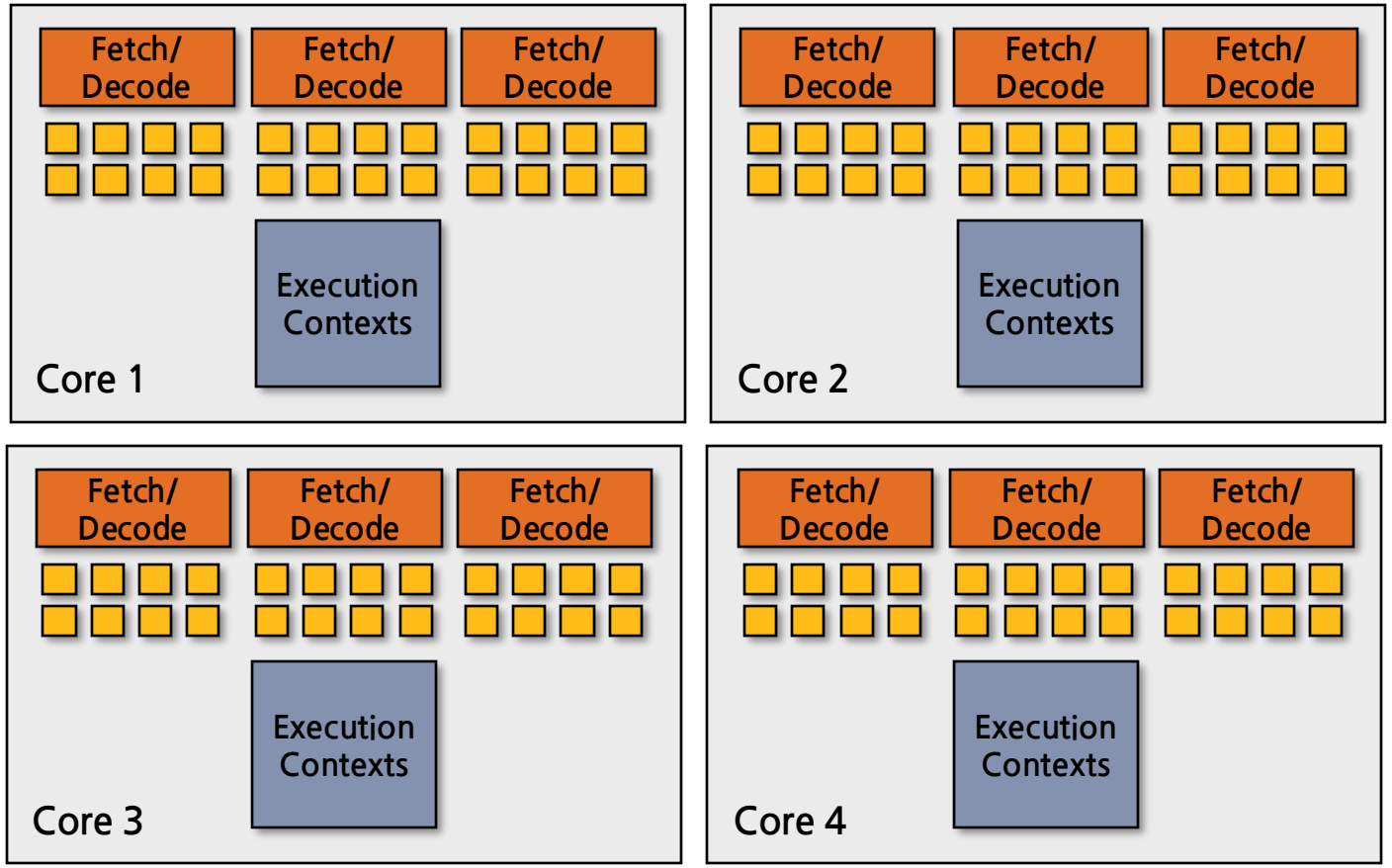
My dual-core processor:
executes one instruction per clock from one instruction stream on each core.



My SIMD quad-core processor:
executes one 8-wide SIMD instruction per clock from one instruction stream on each core.



Example: four-core Intel i7-7700K CPU (Kaby Lake)

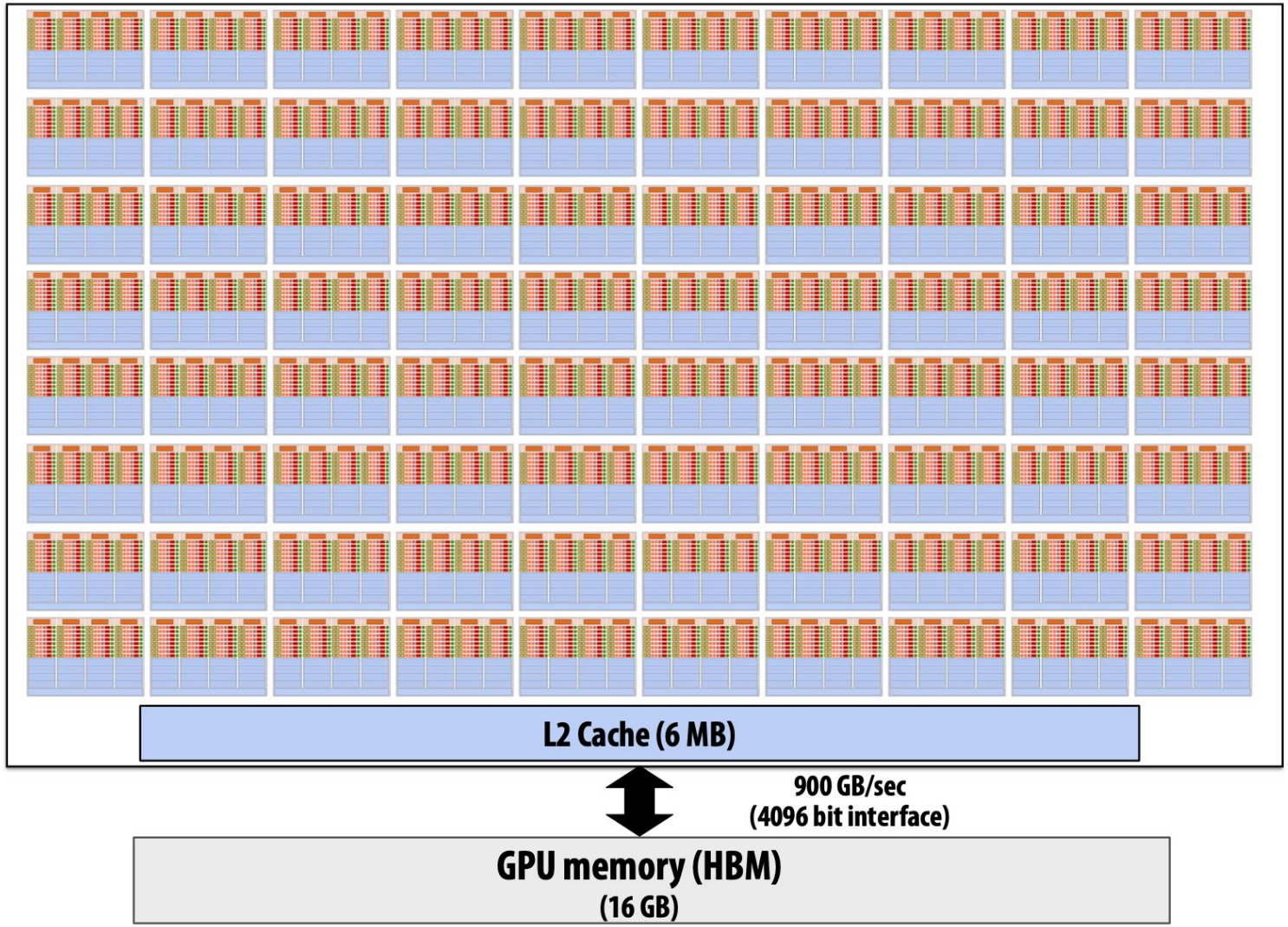


4 core processor
Three 8-wide SIMD ALUs per core
(AVX2 instructions)

4 cores x 8-wide SIMD x 3 x 4.2GHz = 400GFLOPs

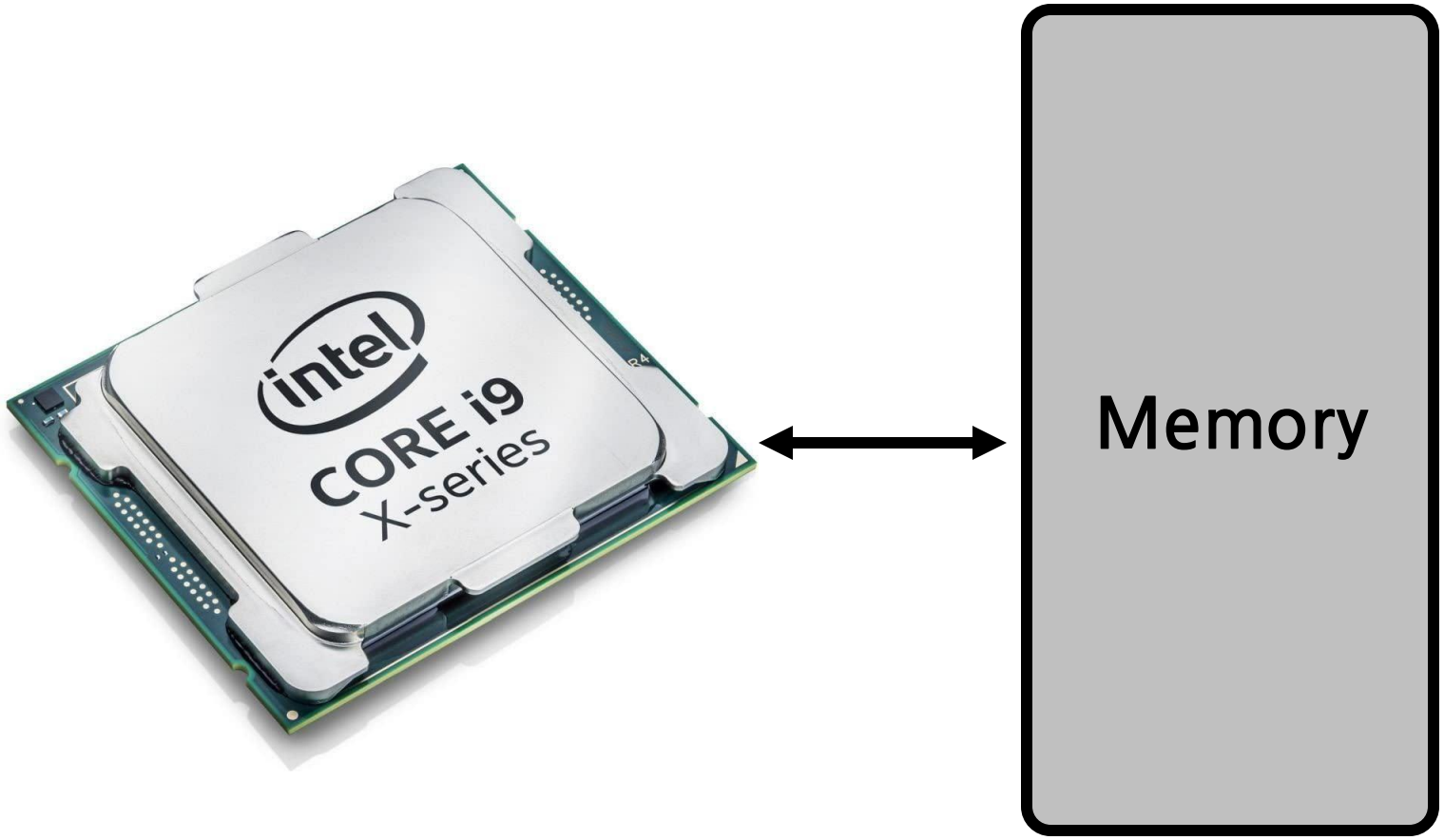
* Showing only AVX math units, and fetch/decode unit for AVX (additional capability for integer math)

NVIDIA V100 GPU (80 SMs)



80 “SM” cores
128 SIMD ALUs per “SM” (@1.6 GHz)
= 16 TFLOPs (~250 Watts)

Part 2: accessing memory



A program's memory address space

- A computer's memory is organized as an array of bytes

- Each byte is identified by its "address" in memory (its position in this array)
  (Today we'll assume memory is byte-addressable)

"The byte stored at address 0x8 has the value 32."

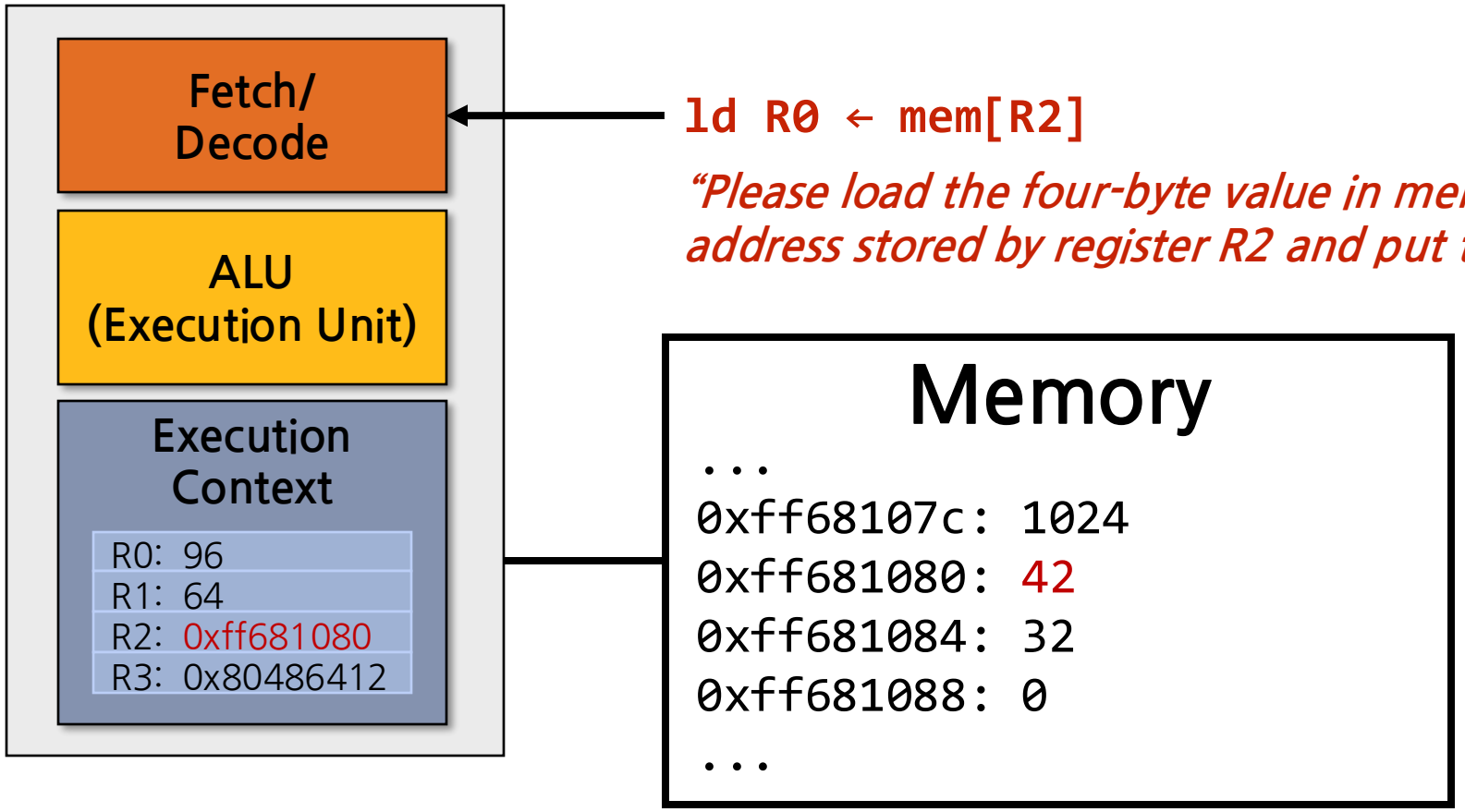
"The byte stored at address 0x10 (16) has the value 128."

In the illustration on the right, the program's memory address space is 32 bytes in size (so valid addresses range from 0x0 to 0x1F)

Address	Value
0x0	16
0x1	256
0x2	14
0x3	0
0x4	0
0x5	0
0x6	6
0x7	0
0x8	32
0x9	48
0xA	255
0xB	255
0xC	255
0xD	0
0xE	0
0xF	0
0x10	128

Load: an instruction for accessing the contents of memory

A Very Simple Processor



`ld R0 ← mem[R2]`

"Please load the four-byte value in memory starting from the address stored by register R2 and put this value into register R0."

Terminology

- Memory access latency
  - The amount of time it takes the memory system to provide data to the processor
  - Example: 100 clock cycles, 100 nsec

Stalls

- A processor “stalls” when it cannot run the next instruction in an instruction stream because of a dependency on a previous instruction that is not yet complete

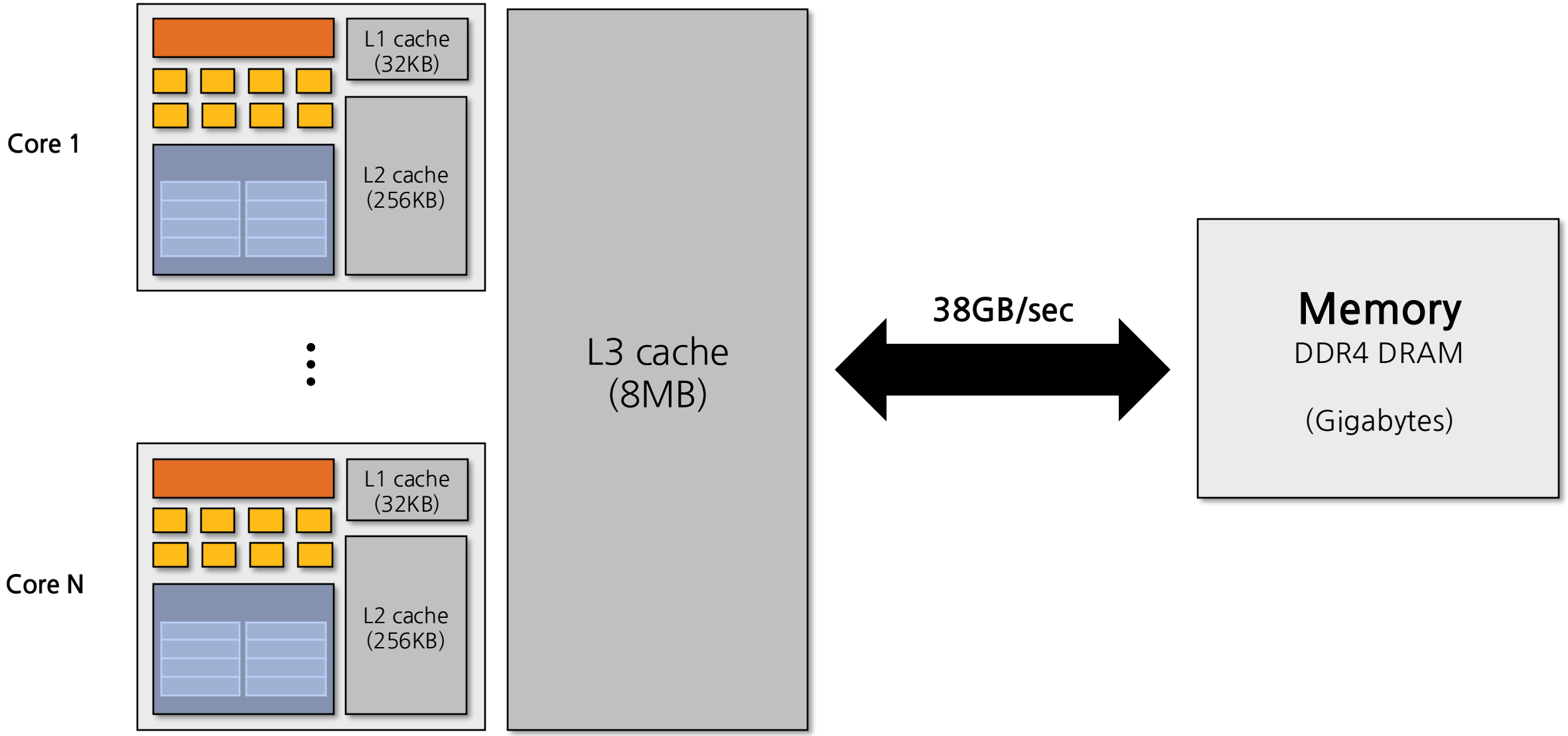
- Accessing memory is a major source of stalls

```
ld r0 mem[r2]
ld r1 mem[r3]
add r0, r0, r1
```

Dependency: cannot execute ‘add’ instruction until data from mem[r2] and mem[r3] have been loaded from memory

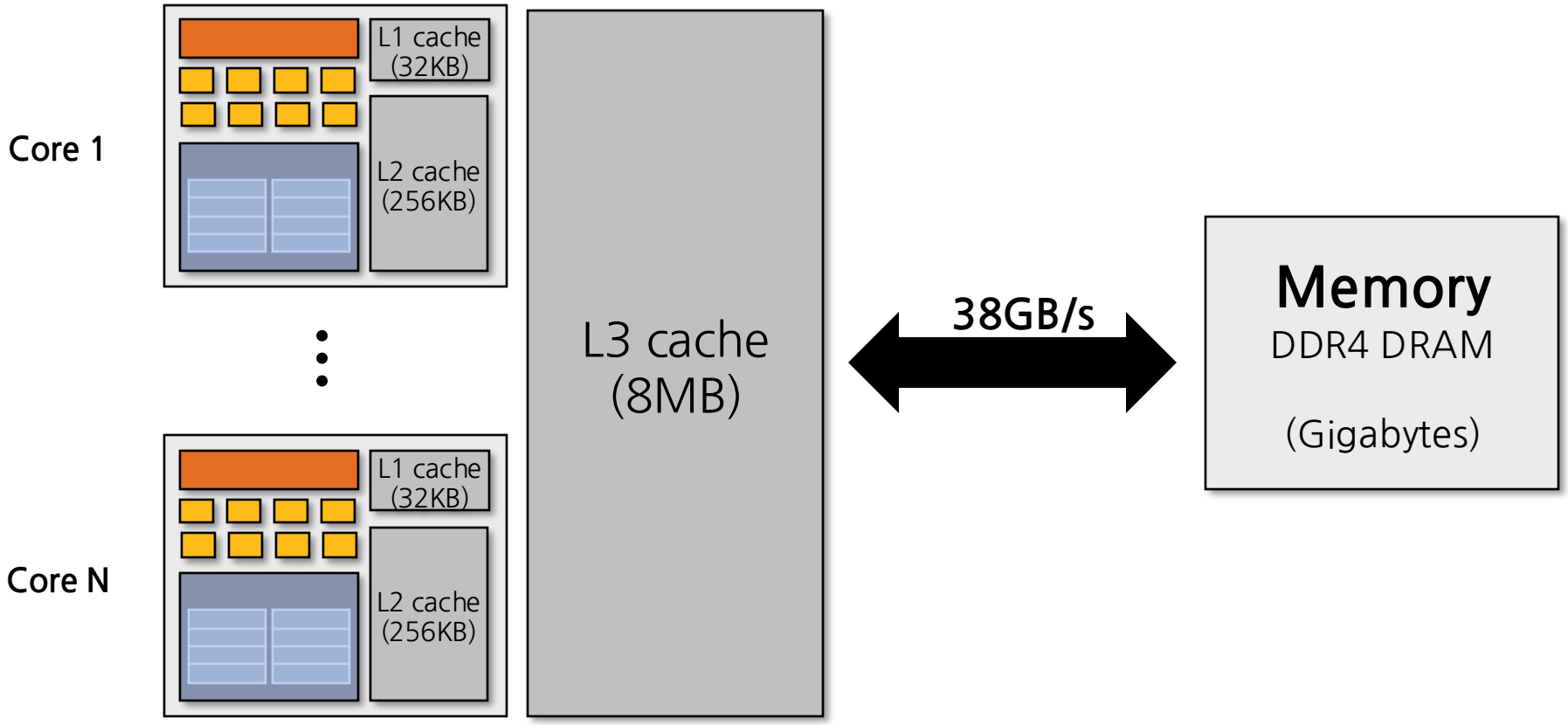
- Memory access times ~ 100’s of cycles
  - Memory “access time” is a measure of latency

Why do modern processors have data caches?



Caches reduce length of stalls

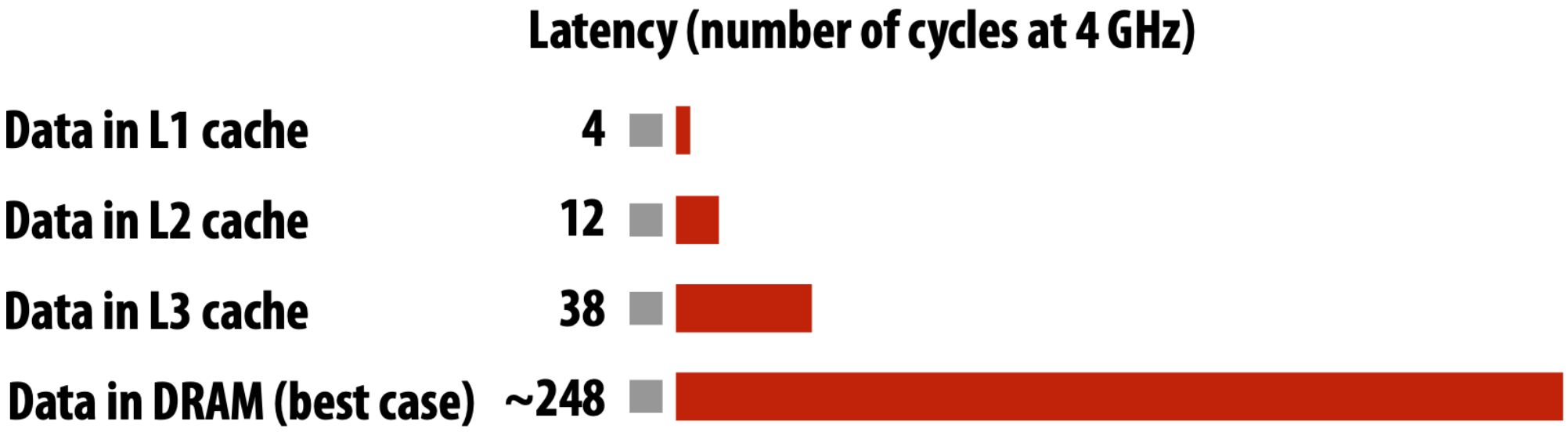
Processors run efficiently when data is resident in caches
Caches reduce memory access latency*



*Caches also provide high bandwidth data transfer to CPU

Data access times

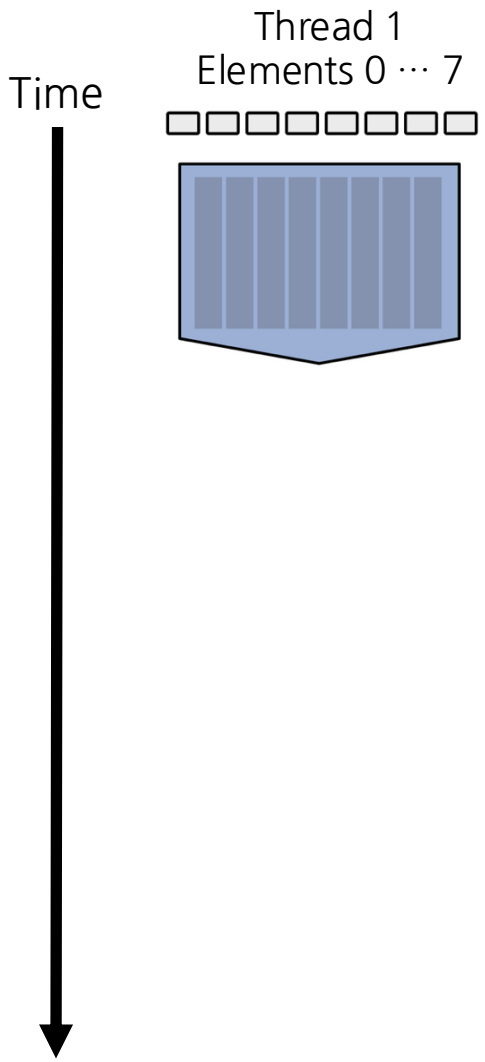
(Kaby Lake CPU)



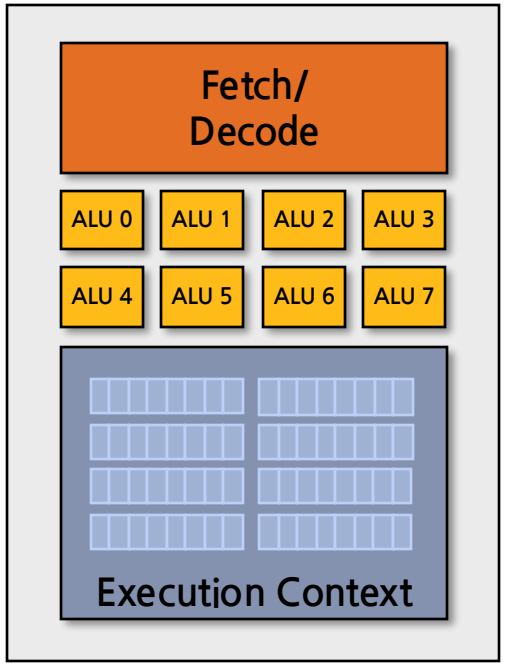
Multi-threading reduces stalls

- Idea #3: interleave processing of multiple threads on the same core to hide stalls
  - If you can’t make progress on the current thread… work on another one

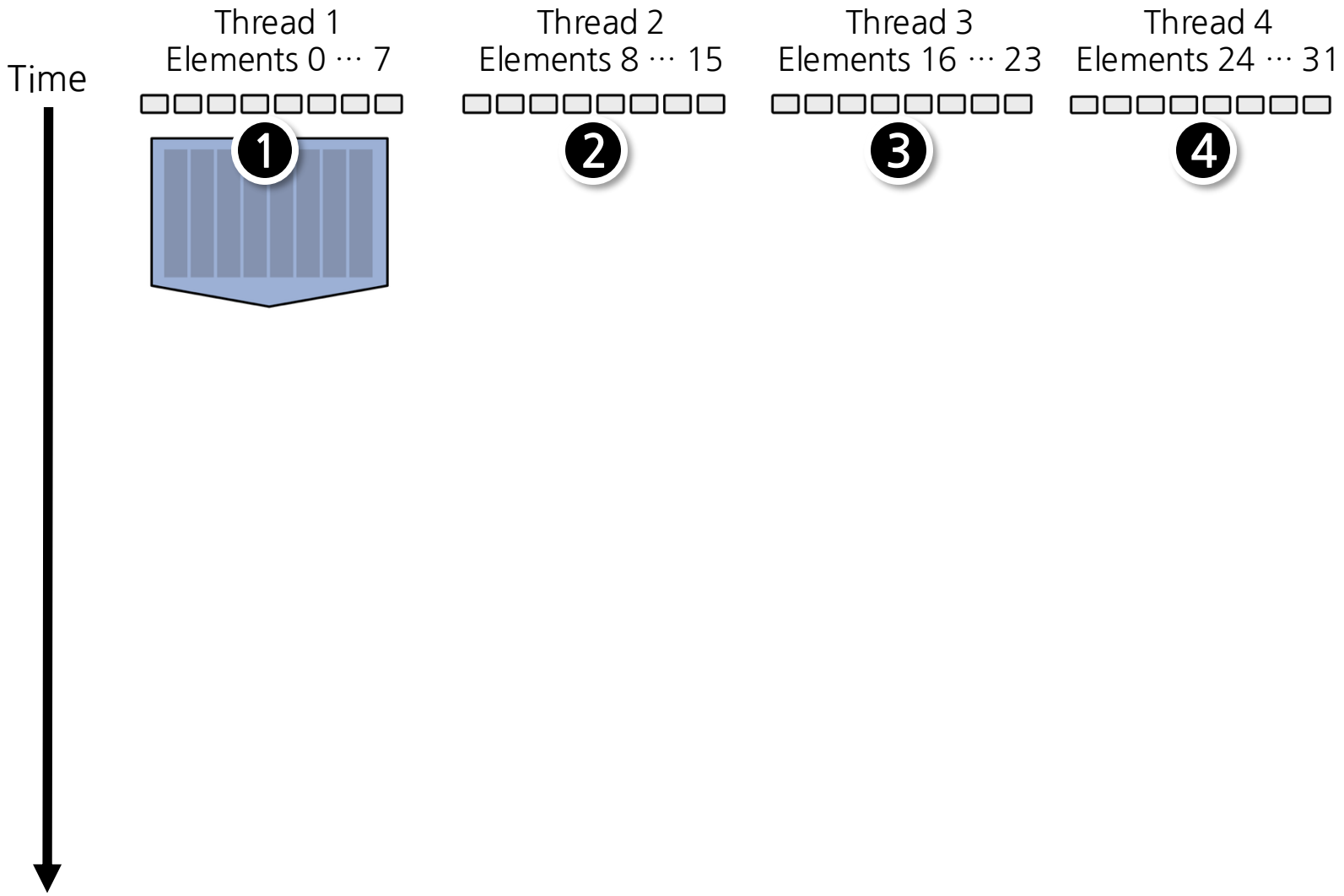
Hiding stalls with multi-threading



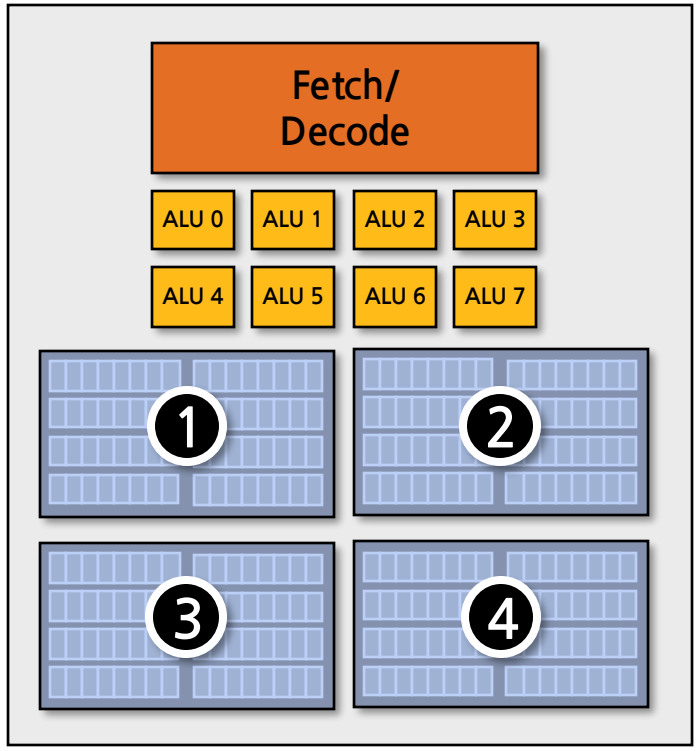
1 Core (1 thread)



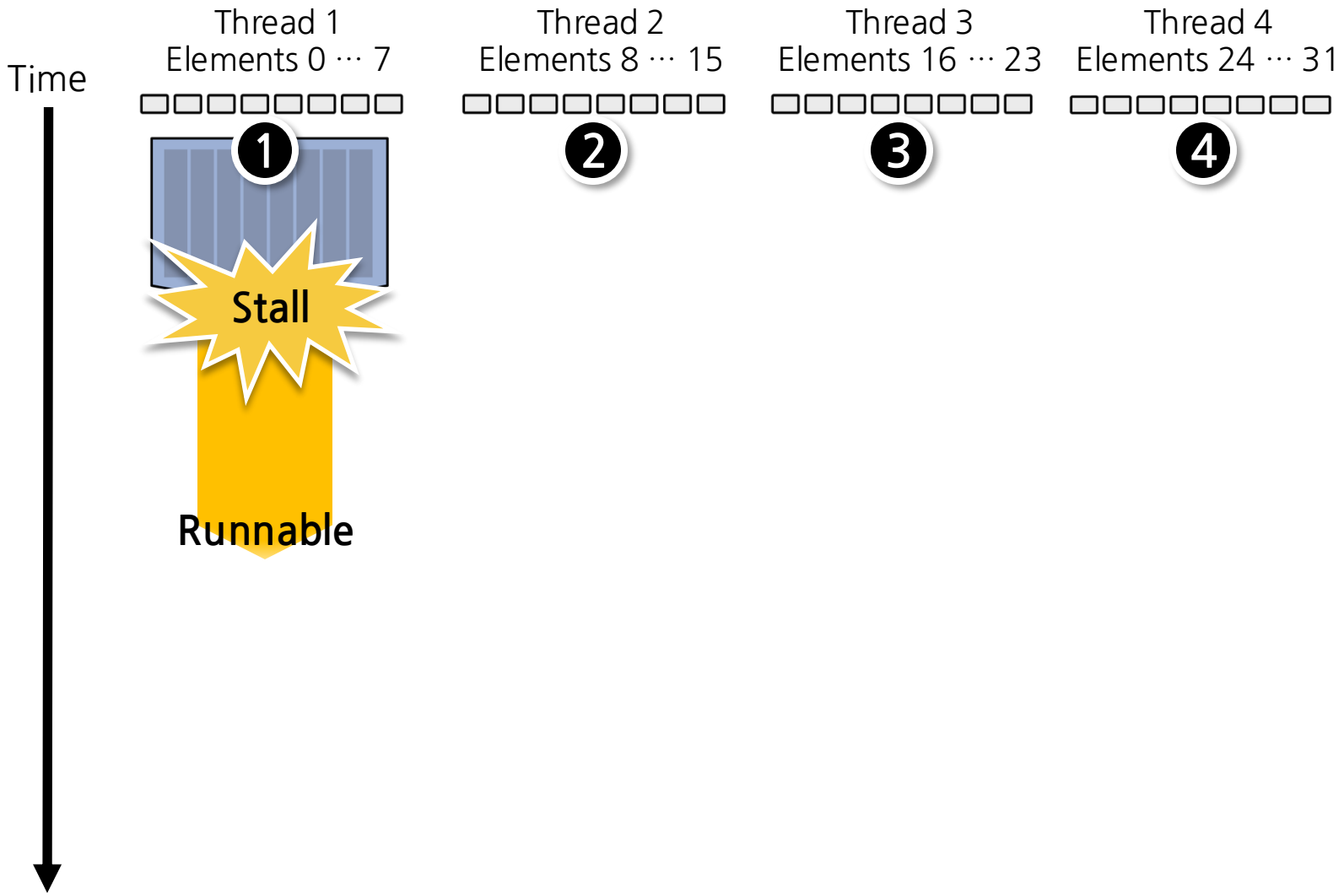
Hiding stalls with multi-threading



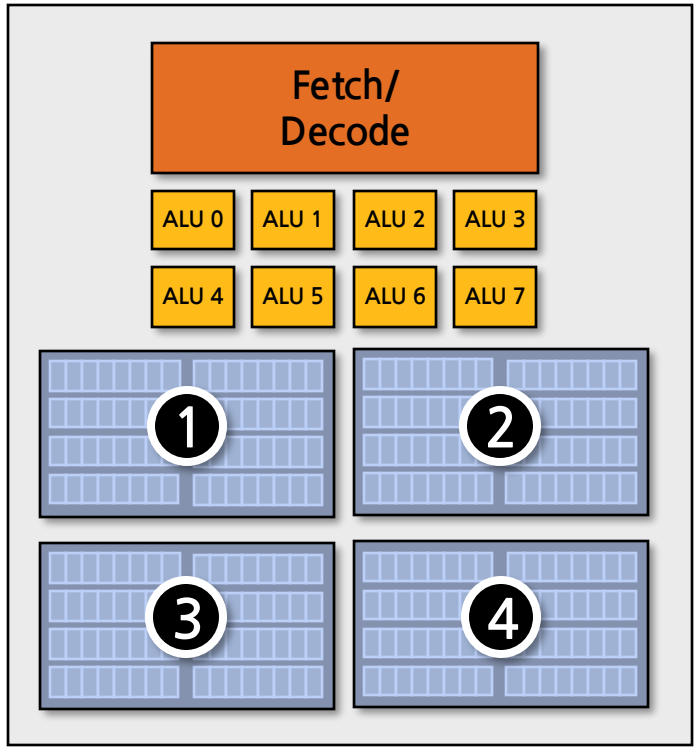
1 Core (4 hardware thread)



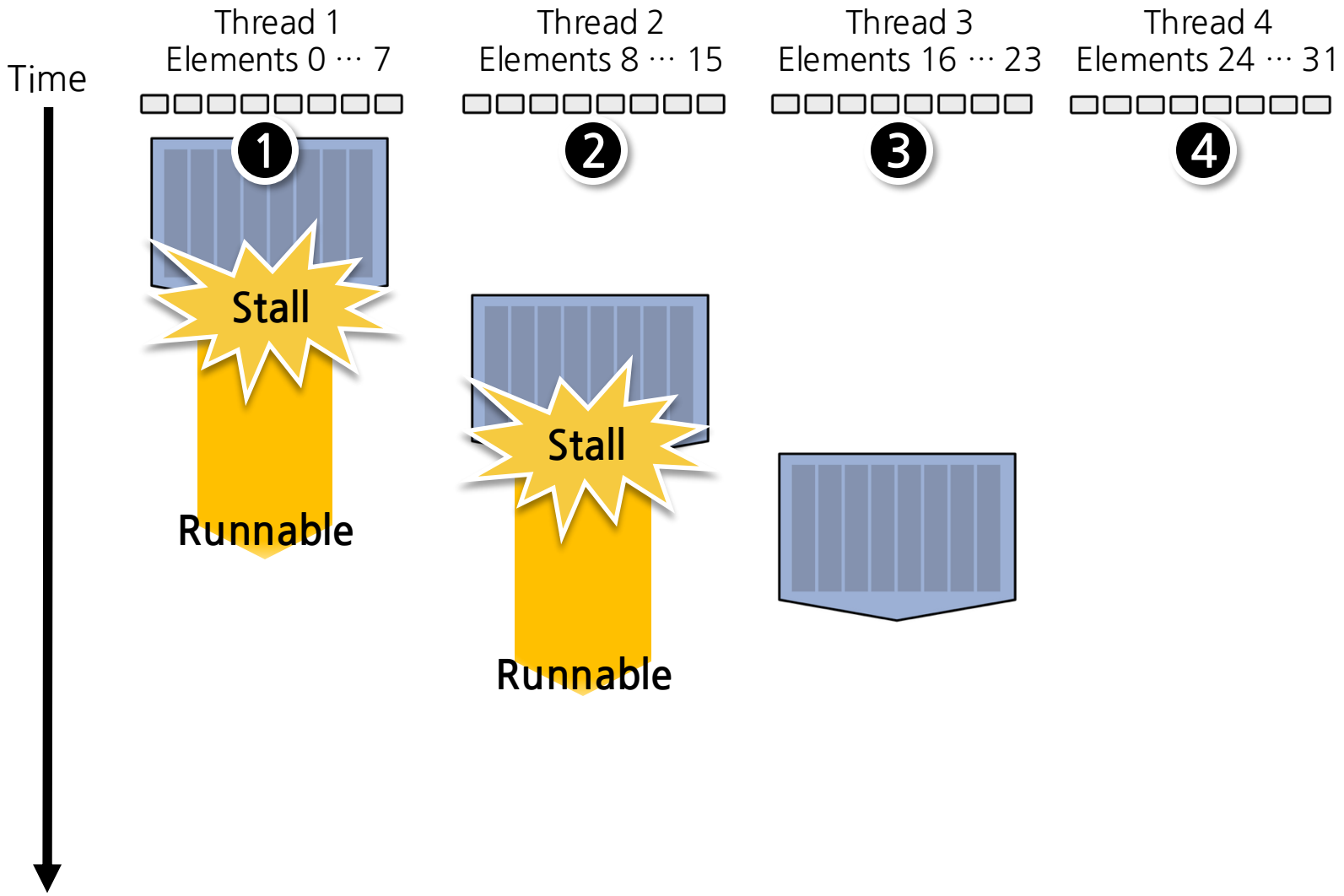
Hiding stalls with multi-threading



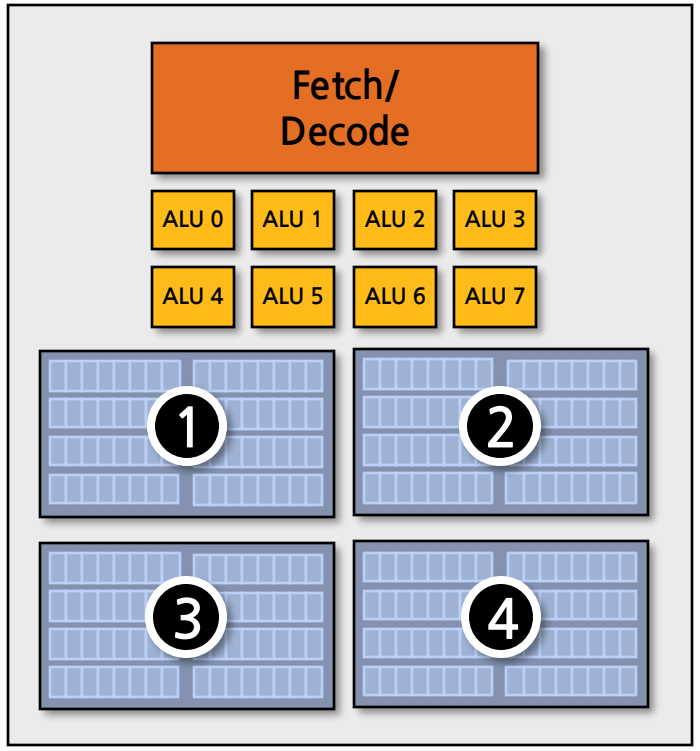
1 Core (4 hardware thread)



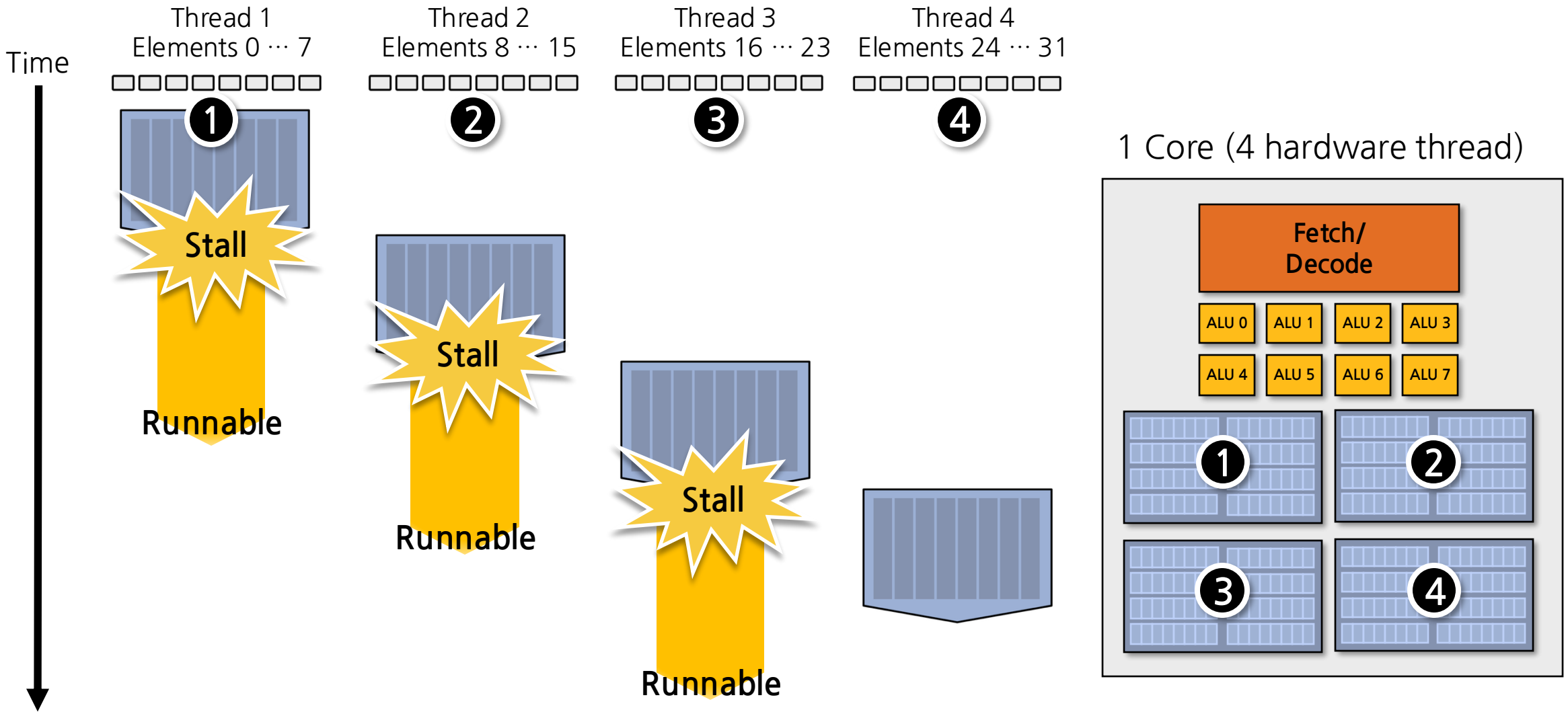
Hiding stalls with multi-threading



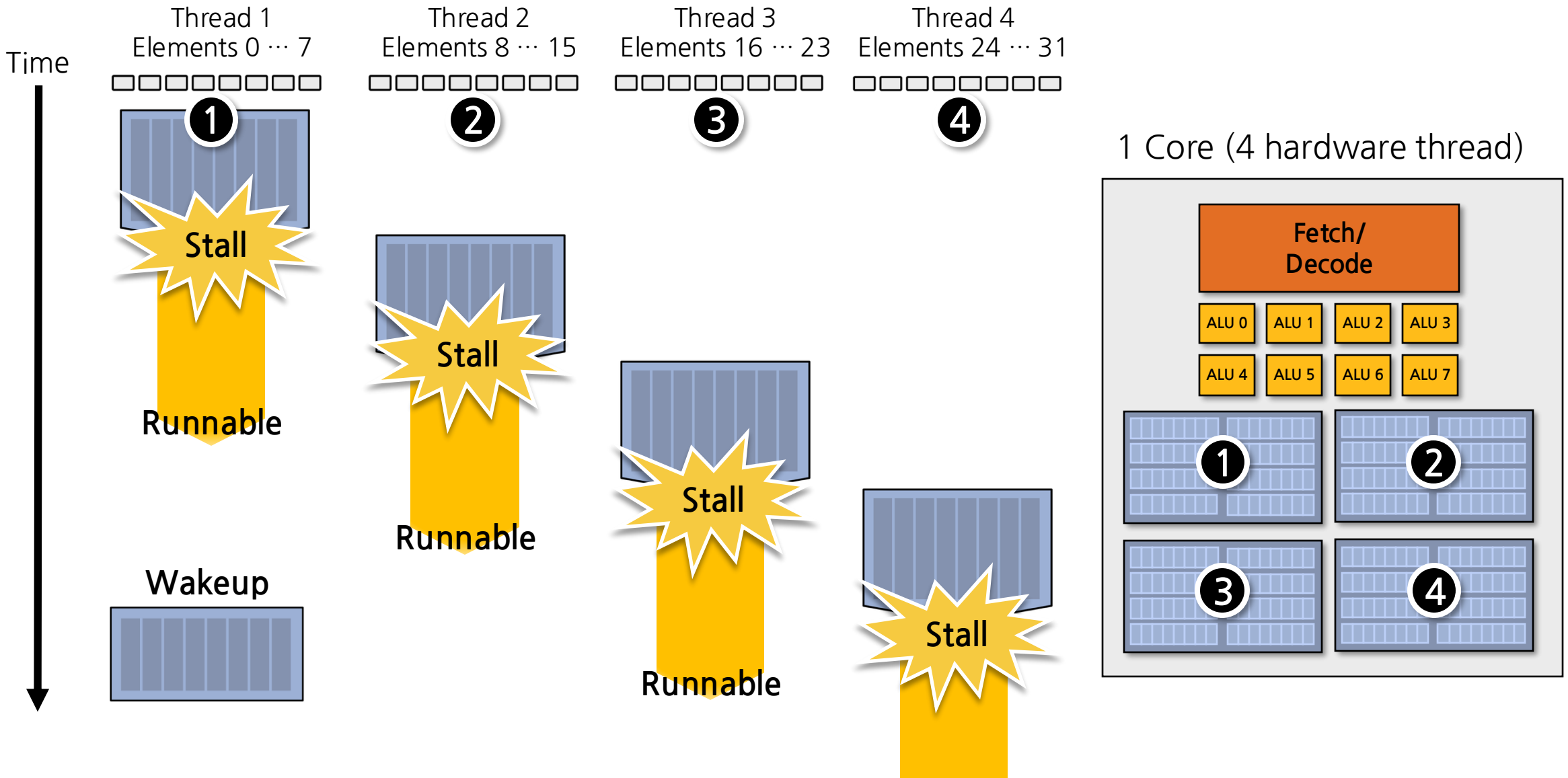
1 Core (4 hardware thread)



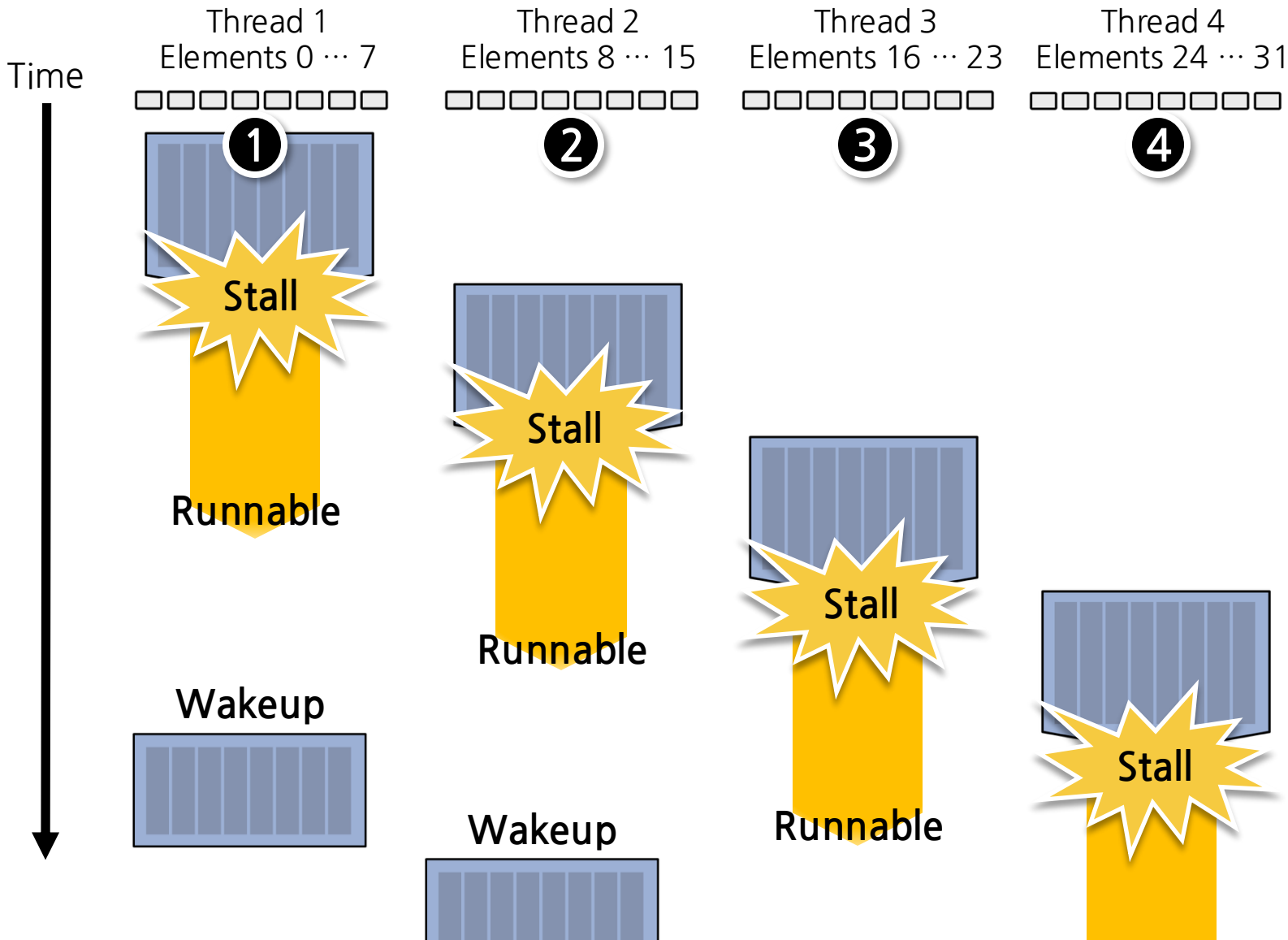
Hiding stalls with multi-threading



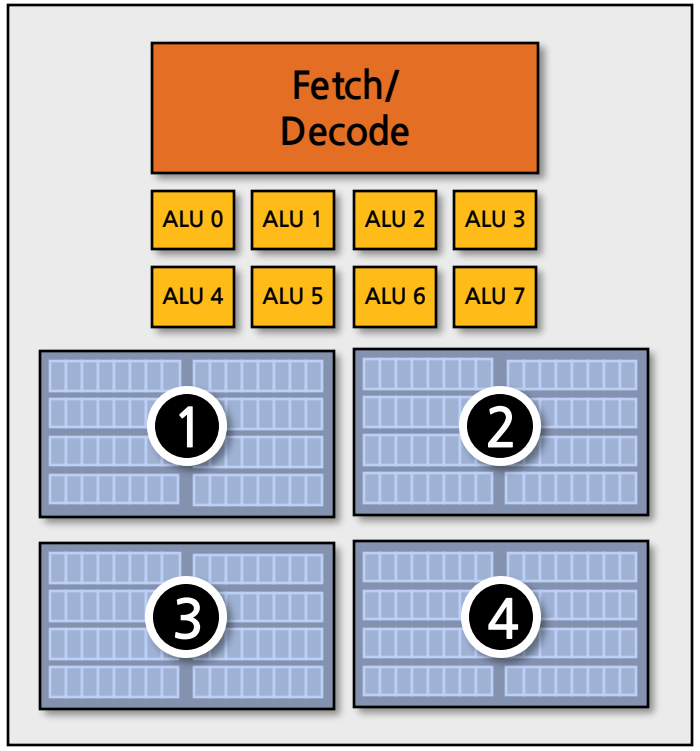
Hiding stalls with multi-threading



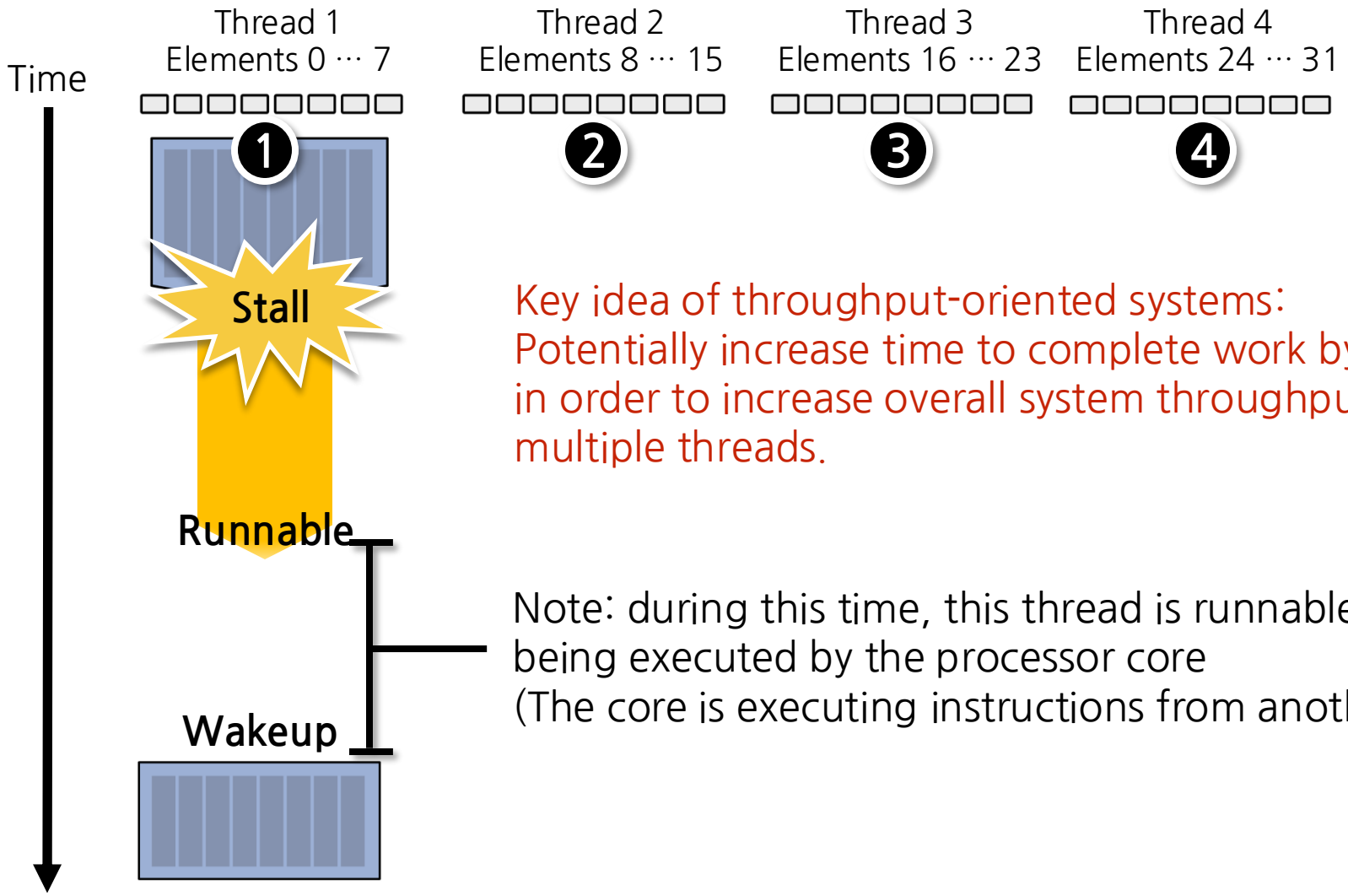
Hiding stalls with multi-threading



1 Core (4 hardware thread)



Throughput computing: a trade-off

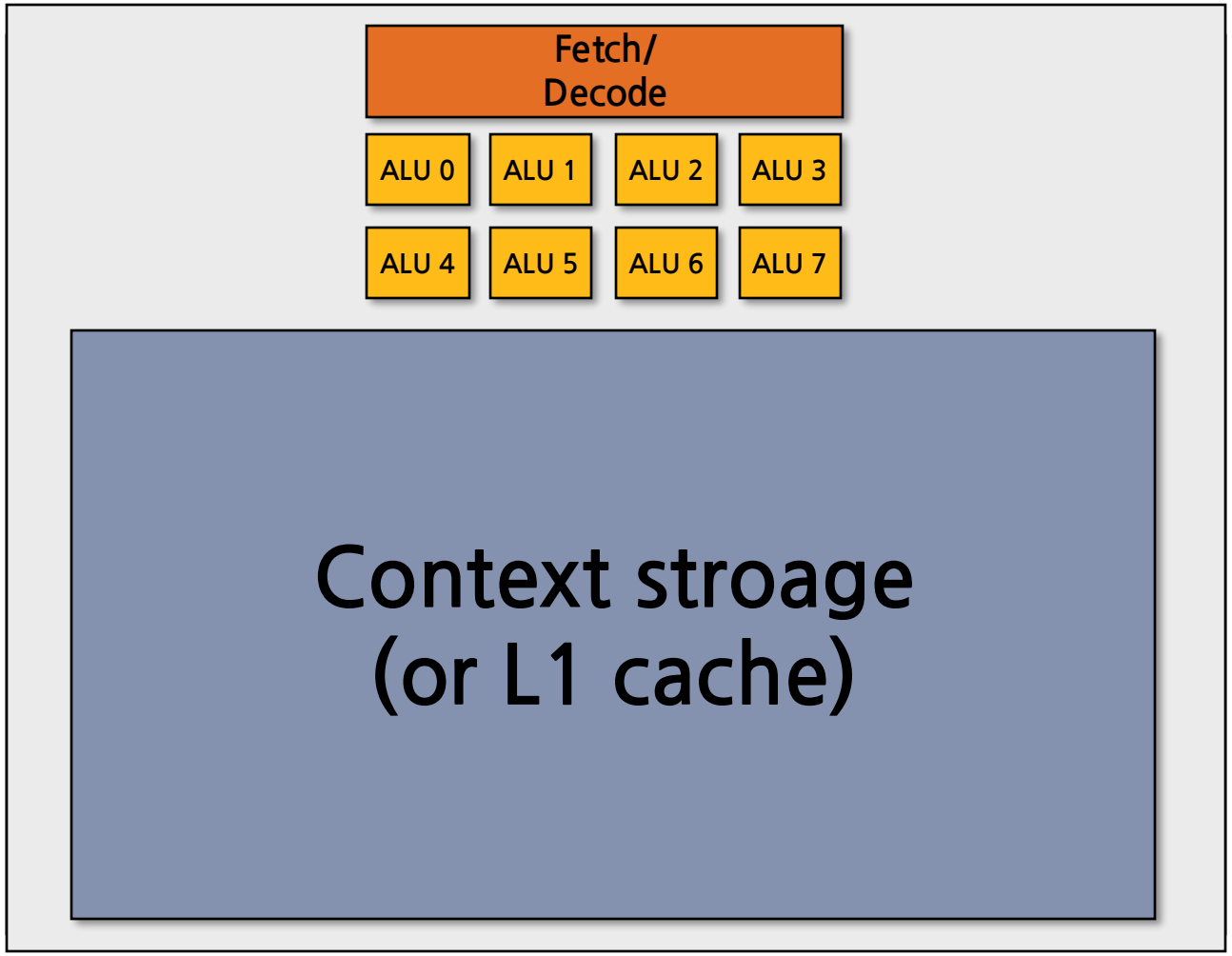


Key idea of throughput-oriented systems:
Potentially increase time to complete work by any one thread, in order to increase overall system throughput when running multiple threads.

Note: during this time, this thread is runnable, but it is not being executed by the processor core
(The core is executing instructions from another thread)

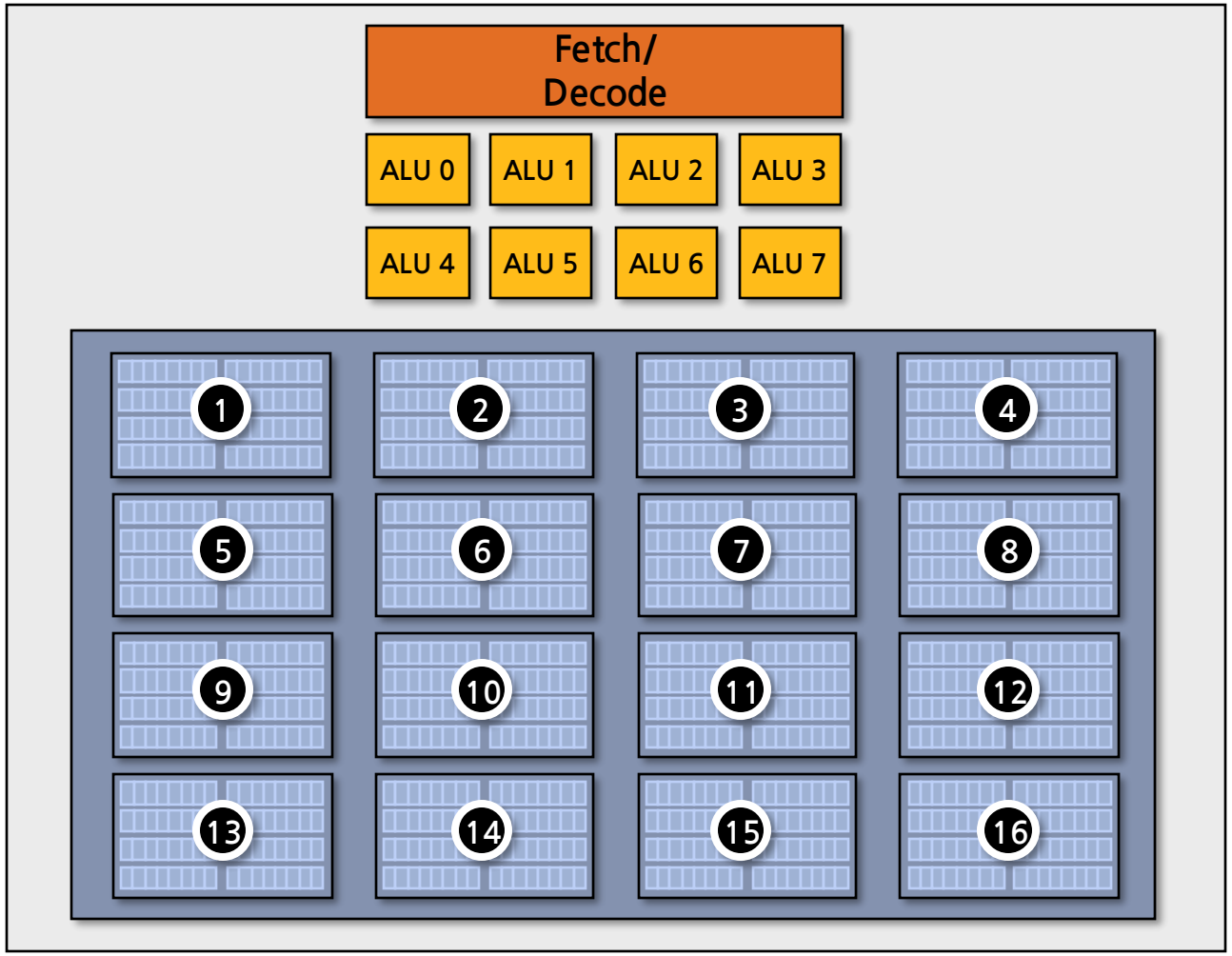
No free lunch: storing execution context

Consider on-chip storage of execution contexts as a finite resource



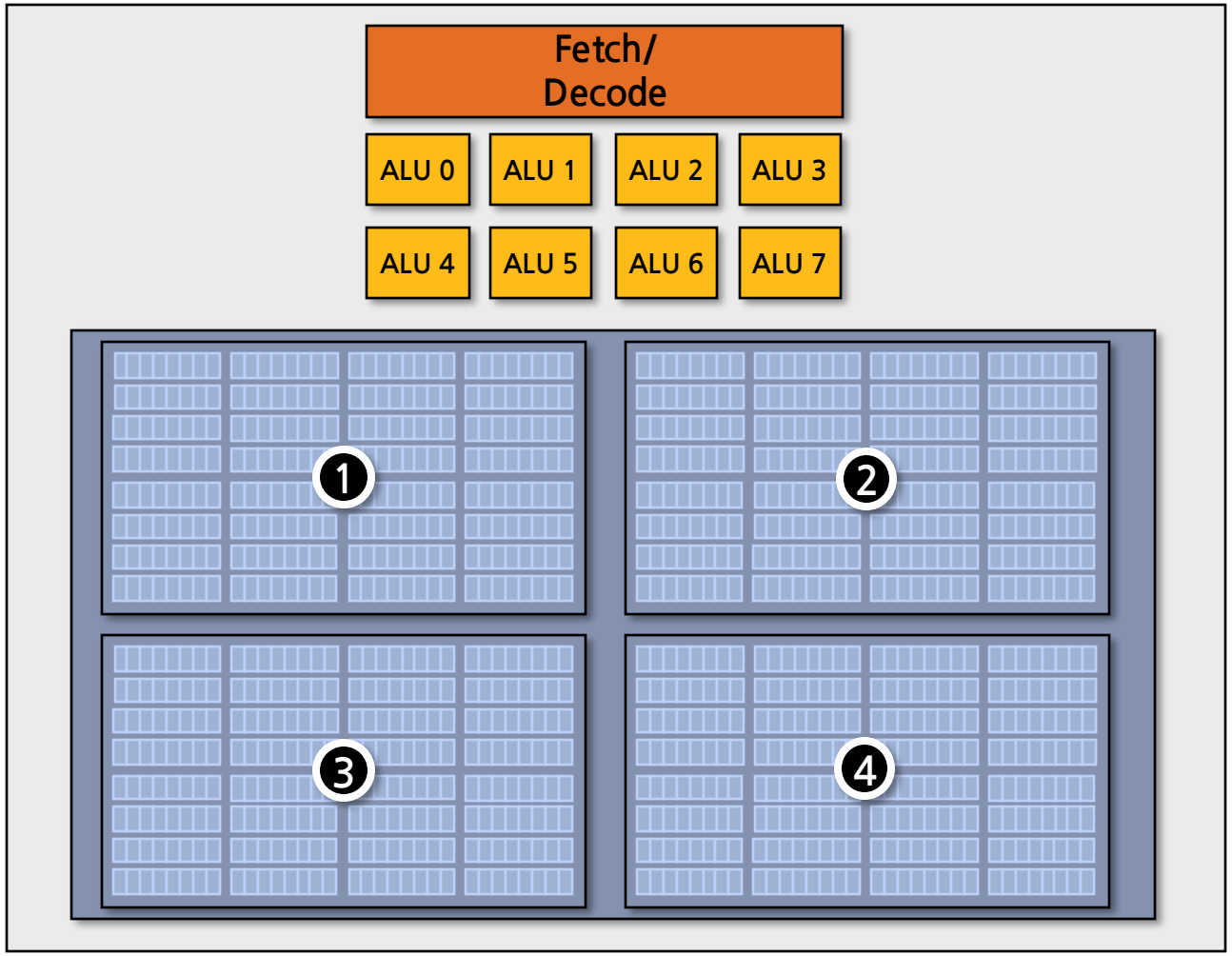
Many small contexts (high latency hiding ability)

16 hardware threads: storage for small working set per thread

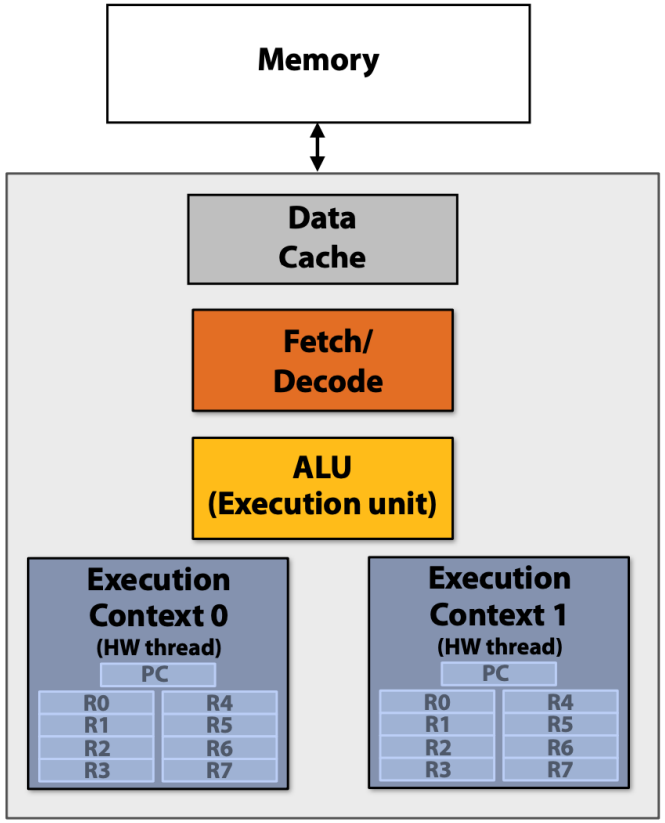


Four large contexts (low latency hiding ability)

4 hardware threads: storage for large working set per thread



Exercise: consider a simple two threaded core

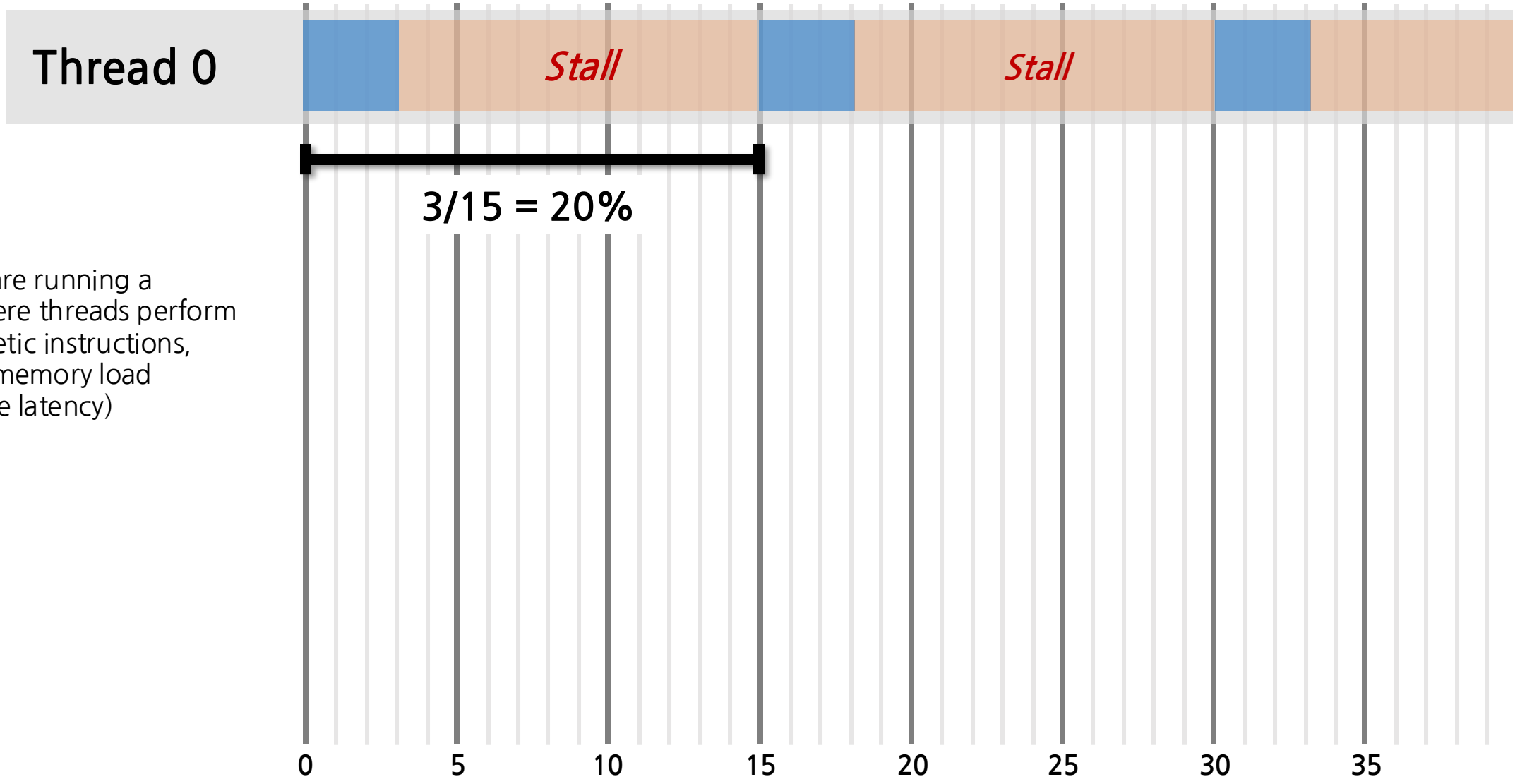


Single core processor, multi-threaded core (2 threads)

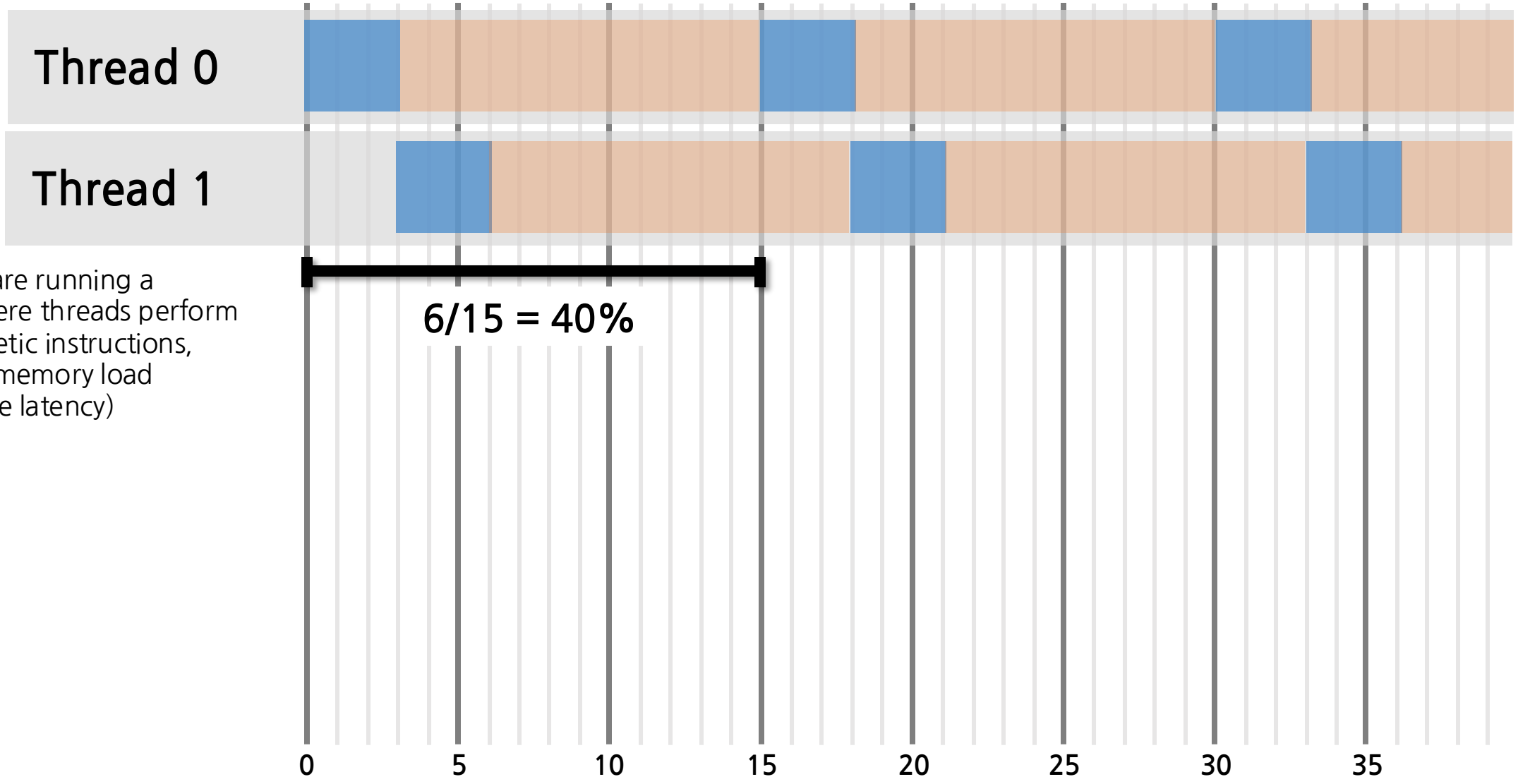
Can run one scalar instruction per clock from
<u>one</u> of the hardware threads

What is the utilization of the core? (one thread)

Assume we are running a program where threads perform three arithmetic instructions, followed by memory load (with 12 cycle latency)

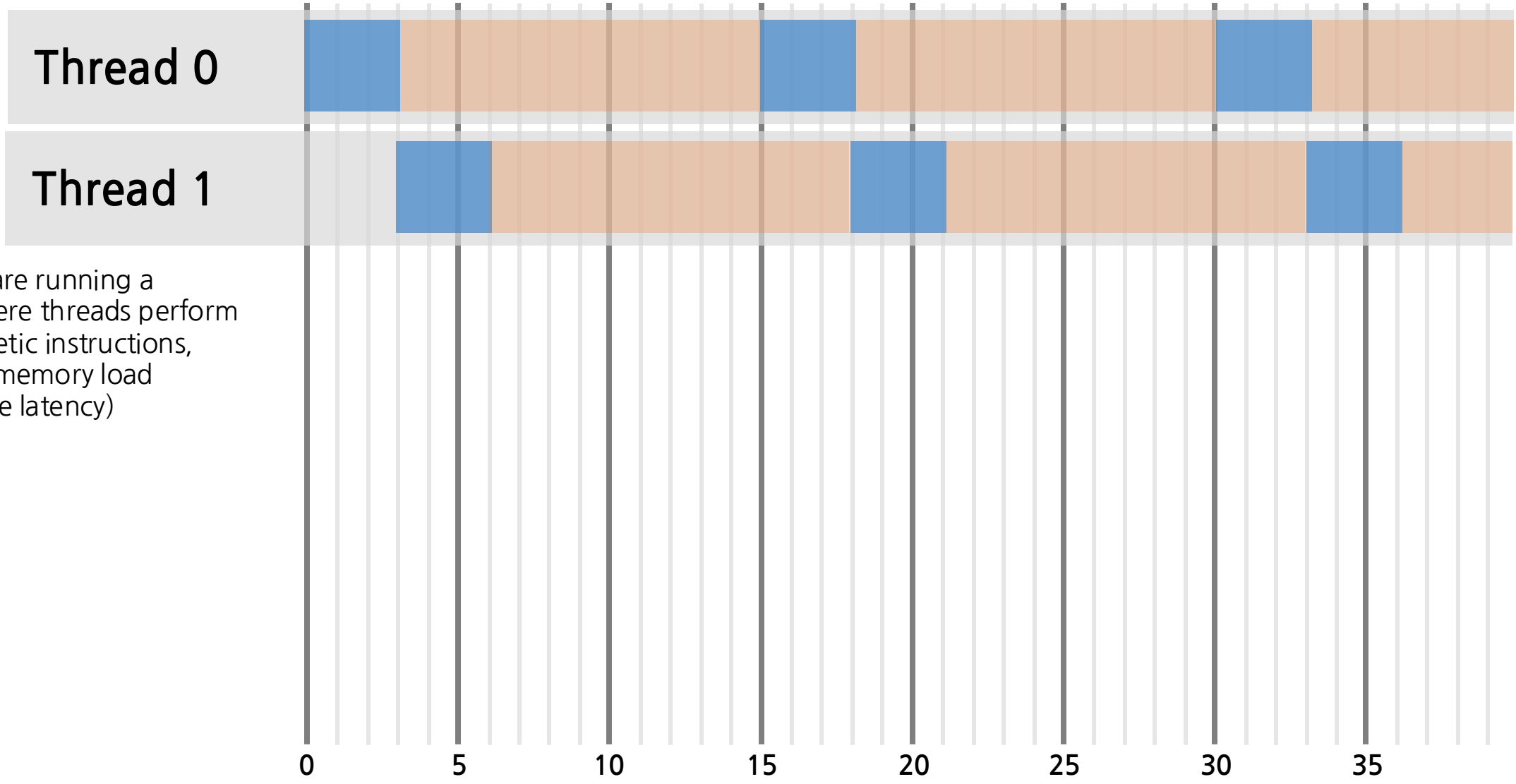


What is the utilization of the core? (two thread)



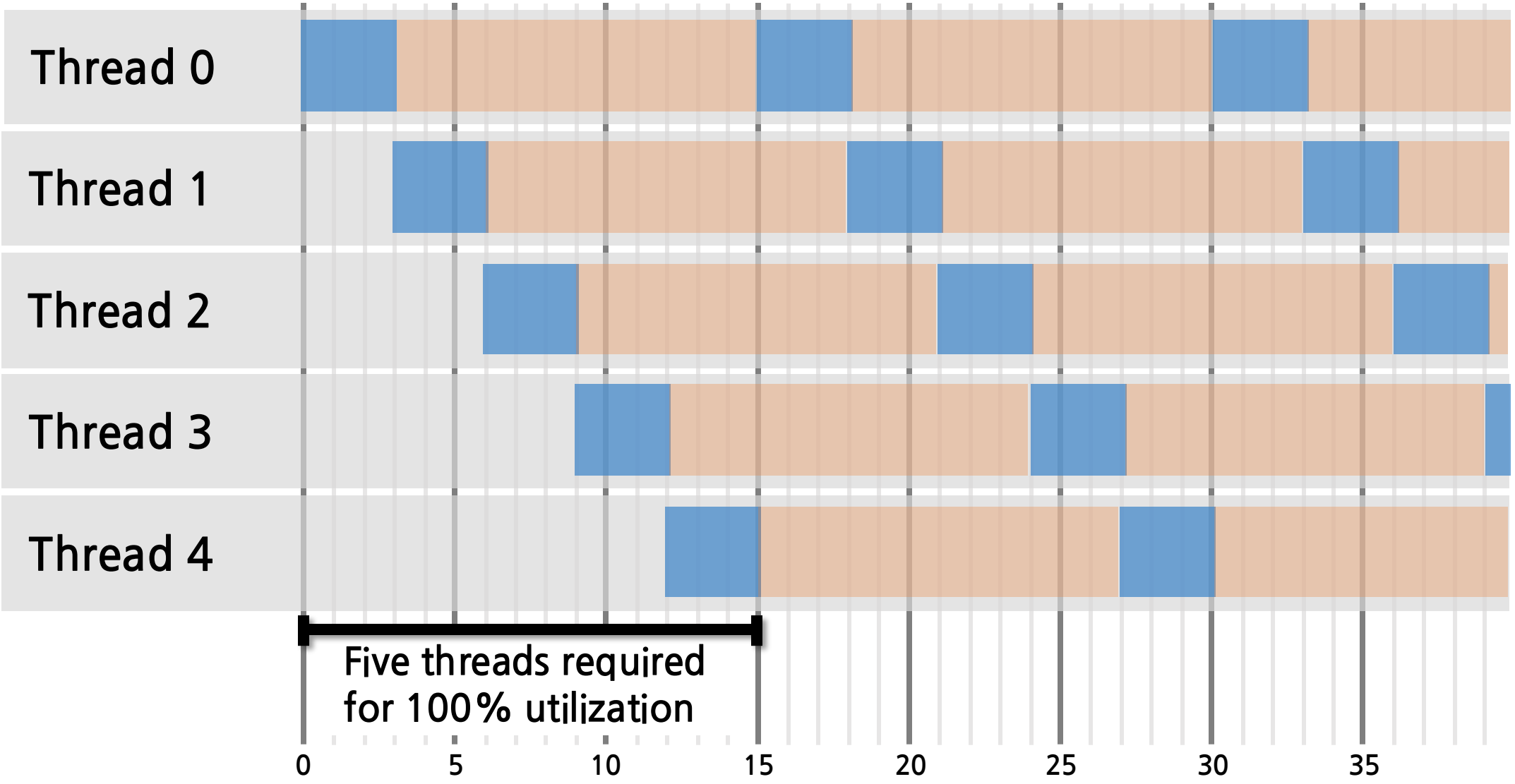
Assume we are running a program where threads perform three arithmetic instructions, followed by memory load (with 12 cycle latency)

How many threads are needed to achieve 100% utilization?

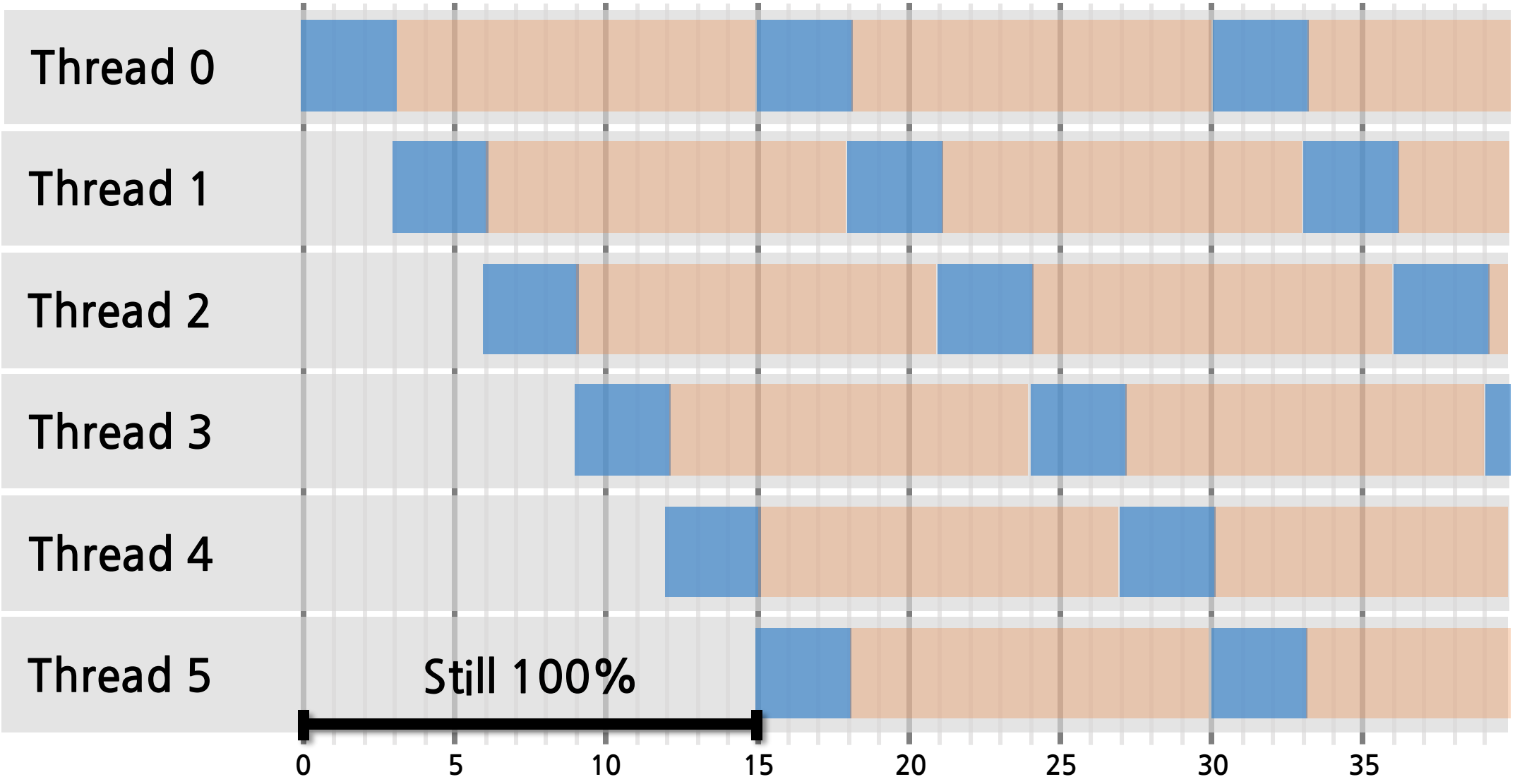


Assume we are running a program where threads perform three arithmetic instructions, followed by memory load (with 12 cycle latency)

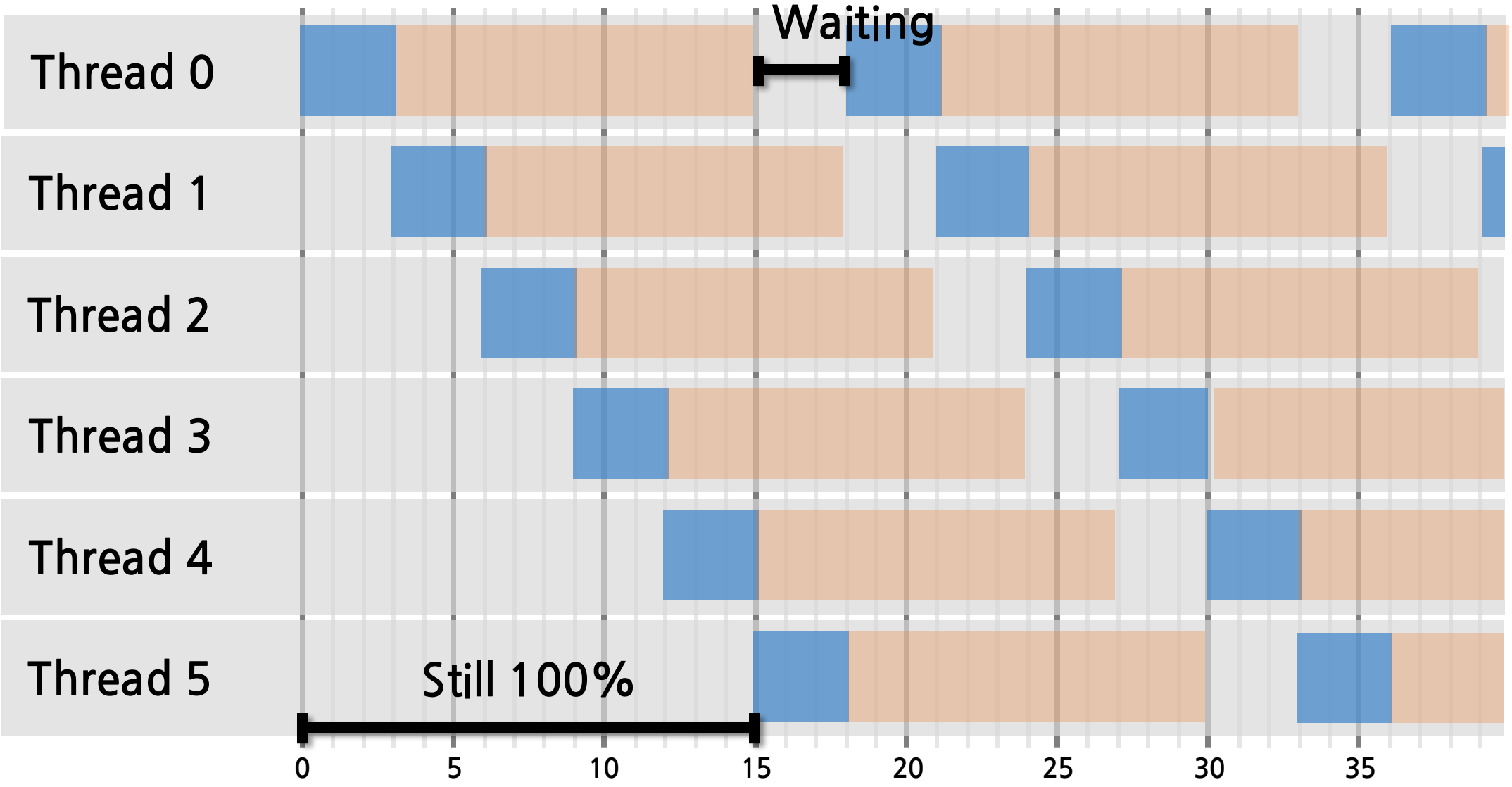
Five threads needed to obtain 100% utilization



Additional threads yield no benefit (already 100% utilization)

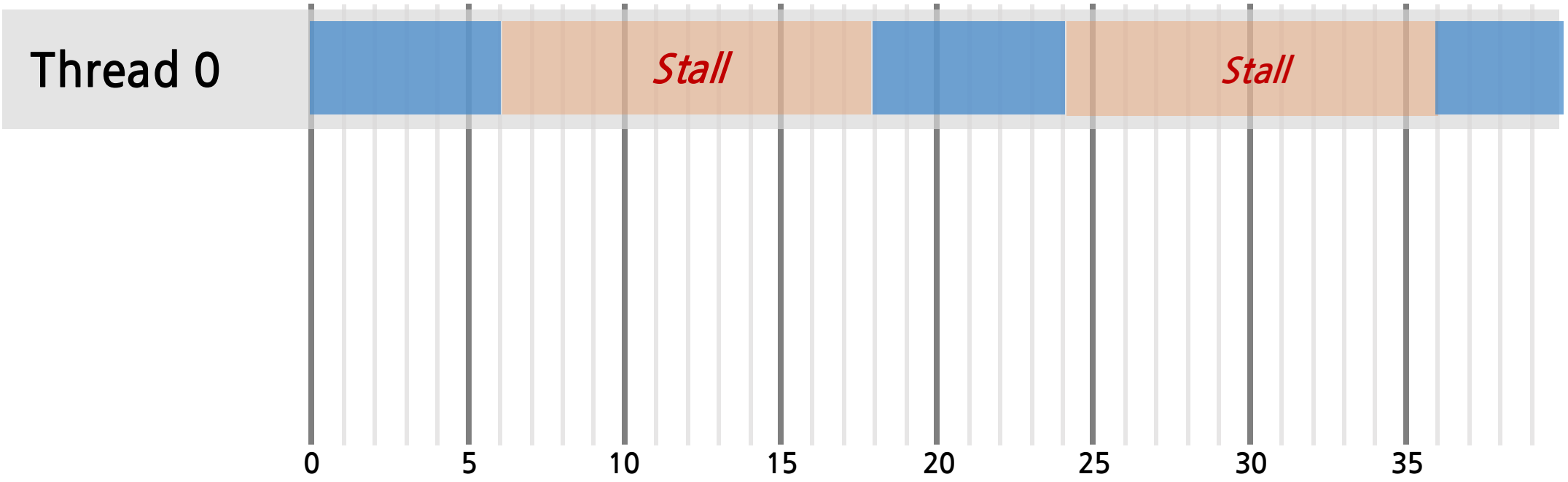


Additional threads yield no benefit (already 100% utilization)



Breakout: How many threads are needed to achieve 100% utilization?

Threads now perform *six arithmetic instructions*, followed by memory load (with 12 cycle latency)



How does a higher ratio of math instructions to memory latency affect the number of threads needed for latency hiding?

Takeaway (point 1)

- A processor with multiple hardware threads has the ability to *avoid stalls* by performing instructions from other threads when one thread must wait for a long latency operation to complete

- Note: the latency of the memory operation is not changed by multi- threading, it just no longer causes reduced processor utilization

Takeaway (point 2)

- A multi-threaded processor hides memory latency by performing arithmetic from other threads

- Programs that feature more arithmetic per memory access need fewer threads to hide memory stalls

Hardware-supported multi-threading

- Core manages execution contexts for multiple threads
  - Core still has the same number of ALU resources: multi-threading only helps use them more efficiently in the face of high-latency operations like memory access
  - Processor makes decision about which thread to run each clock
- Interleaved multi-threading (a.k.a. temporal multi-threading)
  - What I described on the previous slides: each clock, the core chooses a thread, and runs an instruction from the thread on the core's ALUs
- Simultaneous multi-threading (SMT)
  - Each clock, core chooses instructions from multiple threads to run on ALUs
  - Example: Intel Hyper-threading (2 threads per core)

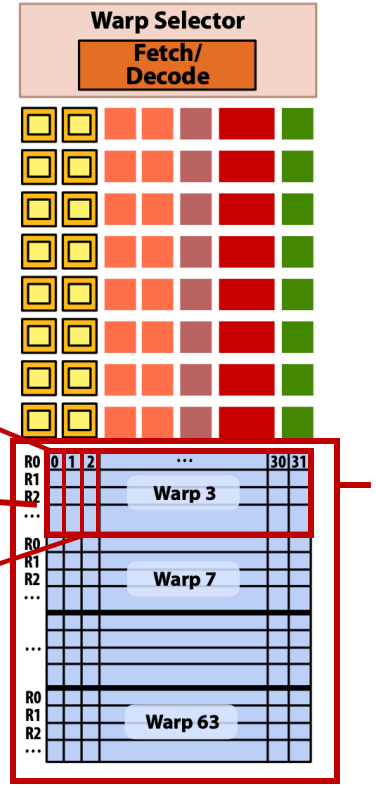
GPUs: extreme throughput-oriented processors

This is one NVIDA V100 streaming multi-processor (SM) unit

Scalar registers for one CUDA thread: R0, R1, etc...

Scalar registers for 32 threads in the same "warp"

Scalar registers for another CUDA thread: R0, R1, etc...



64 KB registers per sub-core

= SIMD fp32 functional unit, control shared across 16 units (16 x MUL-ADD per clock *)

= SIMD int functional unit, control shared across 16 units (16 x MUL/ADD per clock *)

= SIMD fp64 functional unit, control shared across 8 units (8 x MUL/ADD per clock **)

= Tensor core unit

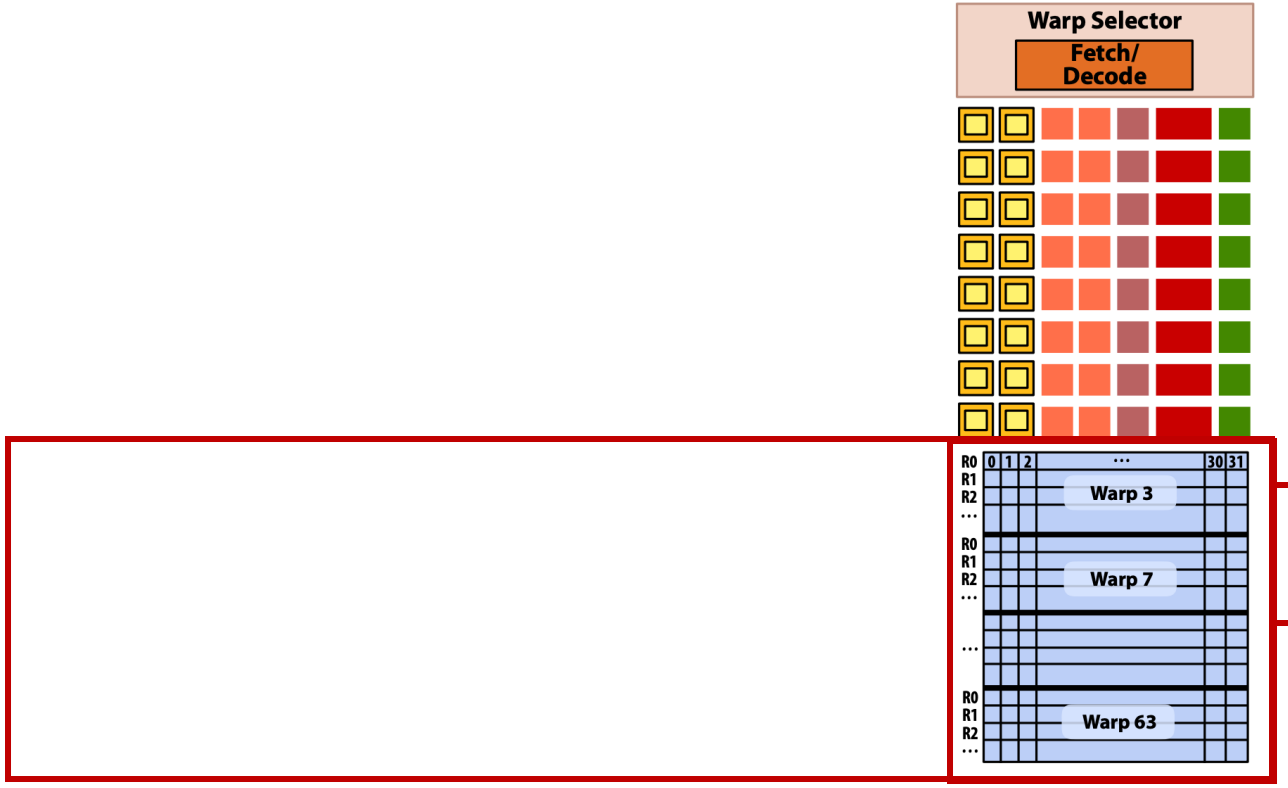
= Load/store unit

* one 32-wide SIMD operation every 2 clocks

** one 32-wide SIMD operation every 4 clocks

GPUs: extreme throughput-oriented processors

This is one NVIDA V100 streaming multi-processor (SM) unit



64 KB registers per sub-core

256 KB registers in total per SM

Registers divided among (up to) 64 “warps” per SM

= SIMD fp32 functional unit, control shared across 16 units (16 x MUL-ADD per clock *)

= SIMD int functional unit, control shared across 16 units (16 x MUL/ADD per clock *)

= SIMD fp64 functional unit, control shared across 8 units (8 x MUL/ADD per clock **)

= Tensor core unit

= Load/store unit

* one 32-wide SIMD operation every 2 clocks

** one 32-wide SIMD operation every 4 clocks

The story so far…

- To utilize modern parallel processors efficiently, an application must:

1. Have sufficient parallel work to utilize all available execution units (across many cores and many execution units per core)

2. Groups of parallel work items must require the same sequences of instructions (to utilize SIMD execution)

3. Expose more parallel work than processor ALUs to enable interleaving of work to hide memory stalls

In modern computing, bandwidth is the critical resource

Performant parallel programs will:

- Organize computation to fetch data from memory less often
  - Reuse data previously loaded by the same thread (temporal locality optimizations)
  - Share data across threads (inter-thread cooperation)
- Favor performing additional arithmetic to storing/reloading values (the math is “free”)
- Main point: programs must access memory infrequently to utilize modern processors efficiently

What we learned today

- Modern parallel processors employ the following throughput computing ideas
  - Use multiple processing cores
    - Simpler cores (embrace parallelism across different threads)
  - Amortize instruction stream processing over many ALUs (SIMD)
    - Increase compute capability with little extra cost
  - Use multi-threading to increase utilization of processing resources
- Due to high arithmetic capability on modern chips, many parallel applications are bandwidth bound